



東南大學

软件设计与体系结构课 实验报告

题 目 架构设计与实现 KWIC 项目

软 件 学 院 院（系） 软 件 工 程 专 业

学 号 71122135

学生姓名 谢鹏宇

指导教师 廖力

起讫日期 2024 年 11 月 8 日—2024 年 11 月 22 日

设计地点 南京

目 录

第一章	主程序-子程序风格	1
1.1	主程序子程序风格设计	1
1.1.1	组件	2
1.1.2	连接件	3
1.2	流程图说明	4
1.3	结果界面截图	4
1.4	风格优缺点	5
1.4.1	优点	5
1.4.2	缺点	6
1.5	总结	7
第二章	批处理风格	8
2.1	批处理风格设计	8
2.1.1	组件	8
2.1.2	连接件	9
2.2	流程图说明	10
2.3	结果界面截图	11
2.4	风格优缺点	11
2.4.1	优点	11
2.4.2	缺点	12
2.5	总结	14
第三章	面向对象风格	15
3.1	面向对象风格设计	15
3.1.1	组件	15
3.1.2	连接件	16
3.2	类图说明	17
3.3	结果界面截图	17
3.4	风格优缺点	18
3.4.1	优点	18
3.4.2	缺点	19
3.5	总结	21
第四章	黑板风格	22
4.1	黑板风格设计	22
4.1.1	组件	23
4.1.2	连接件	24
4.2	流程图说明	25

4.3	结果界面截图	26
4.4	风格优缺点	26
4.4.1	优点	26
4.4.2	缺点	27
4.5	总结	28
第五章	平台/插件风格	29
5.1	平台/插件风格设计	29
5.1.1	组件	30
5.1.2	连接件	31
5.2	流程图说明	31
5.3	结果界面截图	33
5.4	风格优缺点	33
5.4.1	优点	33
5.4.2	缺点	34
5.5	总结	34
第六章	五种风格对比	35
6.1	KWIC 项目中的架构风格比较	35
6.2	特性汇总表	39
6.3	总结	39

第一章 主程序-子程序风格

1.1 主程序子程序风格设计

本章 KWIC 项目采用主程序-子程序风格设计，整个程序由一个主函数 `main()` 和一系列子函数组成，每个子函数独立负责特定的任务，而主函数的职责是协调这些子函数的执行，并通过明确的调用顺序控制整个程序的运行。主程序的调用流程如图 1.2 所示，展示了主程序如何依次调用子函数，完成从输入到输出的全过程。

这种架构的核心思想是将复杂的系统功能分解为多个较小的子任务，并将这些子任务分别在每个独立的子程序中实现。这种功能分解使得每个子程序的职责单一，专注于完成一个明确的任务，如数据输入、移位处理、排序以及结果输出。

主程序-子程序架构的设计不仅确保了任务的顺序执行，还利用共享的数据存储来实现模块之间的数据传递。输入模块将用户提供的原始数据存储在一个共享的存储空间中，移位模块从中读取数据并生成循环移位的结果，排序模块对移位结果进行排序，最后输出模块读取排序后的结果并输出到用户界面。

通过这种方式，系统被分解为多个独立的功能模块，主要包括以下四个模块：

- 输入模块：负责从用户获取输入并将其存储。
- 移位模块：基于输入的数据生成所有可能的循环移位。
- 排序模块：对生成的循环移位结果按照字母顺序进行排序。
- 输出模块：将排序后的结果展示给用户。

主程序严格按照这一逻辑顺序依次调用这些模块，确保数据的有序处理，最终实现 KWIC 功能的完整流程。模块之间的协作通过共享的数据存储实现，减少了模块间的复杂依赖，同时也提供了较高的性能。主程序-子程序架构的设计图如图 1.1 所示，直观地展示了模块之间的交互及数据流动方式。

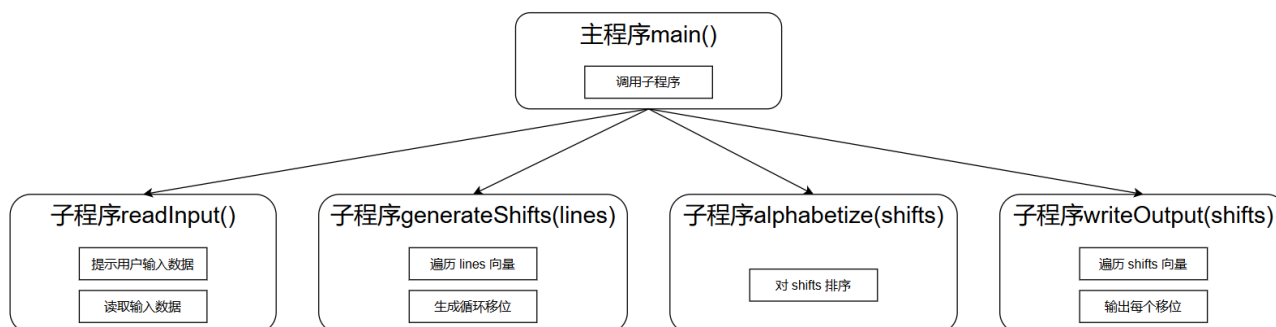


Figure 1.1 主程序子程序风格实现 KWIC 设计图

1.1.1 组件

在主程序-子程序架构中，每个子程序被设计为一个独立的组件，专注于完成特定的任务。这些组件之间通过明确的接口和职责划分进行协作。程序的核心功能被分解为输入、移位、排序和输出四个部分，每个组件实现一个特定的功能模块。通过这种方式，程序实现了功能的模块化和流程的清晰化，为整个系统的稳定性和可维护性奠定了基础。

(1) 读取用户输入 **readInput**

该组件的主要职责是从用户输入中逐行读取文本内容，并将这些输入整理成结构化的数据形式供后续处理。用户通过控制台逐行输入文本，每次输入的一行文本会被暂时存储在一个字符串变量中，随后依次存入一个字符串向量（`vector<string>`）中。当用户输入关键字“END”作为结束信号后，组件停止读取，并将累积的文本行向量返回给主程序。

该组件作为数据流的起点，承担着将非结构化的用户输入转化为可以程序化处理的数据结构的任务。这一过程确保了用户输入的多样性能够被统一存储和处理，为后续的移位和排序逻辑提供了规范化的输入数据。通过这种设计，程序的输入部分与处理部分实现了解耦，既提高了系统的灵活性，也便于后续模块的开发和维护。

(2) 生成循环移位 **generateShifts**

该组件负责接收来自 **readInput** 组件的字符串向量作为输入，并对这些输入中的每一行文本生成所有可能的循环移位。循环移位的核心逻辑是将文本行中的单词重新排列，每次将一个单词作为起始位置，其余单词按顺序接续，直到所有可能的排列组合被生成完毕。每一组循环移位结果被存储为一个字符串，并依次添加到一个新的字符串向量中，最终返回包含所有移位结果的完整数据结构（`vector<string>`）。

该组件的主要职责是将输入文本行转换为循环移位形式的排列结果，为后续排序处理提供多样化的数据来源。通过这种处理方式，系统能够生成全面的文本排列组合，提升了数据的丰富性，同时确保了每一行输入文本都能够被充分分析和处理。生成的移位数据也体现了系统的核心处理逻辑，是整个功能实现的关键环节。

(3) 对移位结果排序 **alphabetize**

该组件接收来自 **generateShifts** 组件的循环移位结果，并对这些结果按照字母顺序进行排序。排序的实现基于自定义的比较函数，将未排序的字符串向量中的元素直接进行升序排列，排序结果存储在原有的数据结构中，无需额外的空间开销。排序完成后，循环移位结果的排列顺序得到规范化，形成按字母顺序排列的文本集。

该组件的主要职责是对未排序的循环移位数据进行排序处理，为最终结果的展示提供规范化的数据输出。通过对结果进行字母顺序排列，用户能够更加直观地理解和分析文本的内容和排列方式。排序逻辑的独立封装也使得系统具备较高的扩展性，便于在未来引入其他排序方式或优化算法。

(4) 输出结果 **writeOutput**

该组件负责接收来自 **alphabetize** 组件的排序结果，并将这些结果逐行输出到控制台，供用户查看。组件通过遍历排序后的字符串向量，依次将每一条结果按照其排列顺序打印到标准输出中。输出的内容已经过循环移位和排序的完整处理，直观地反映了程序的核

心功能。用户可以通过控制台清晰地看到最终的结果，这些结果以字母顺序排列，易于分析和理解。

该组件作为程序的输出终端，承担着将处理完成的数据展示给用户的任务。其主要职责是确保用户能够方便地获取程序的处理结果，同时直观地感受到系统的功能效果。通过这一组件，整个数据处理流程得以完整闭环，将结果从程序内部传递到用户可见的界面。

每个组件都承担了一个明确的职责，并通过标准化的接口和明确的输入输出与其他组件协作。这种模块化的设计使得系统具有较高的可扩展性和可维护性，便于独立开发和调试各个模块。此外，组件的职责分离减少了相互依赖，提高了系统的稳定性和可复用性。

1.1.2 连接件

连接件是组件之间的桥梁，负责在各功能模块之间传递数据。在主程序-子程序架构中，数据在组件之间的流动通过连接件实现，确保了各个组件之间的协作。连接件的设计不仅使数据流向清晰，还使得程序逻辑更为直观，保证了整个系统的顺畅运行。

(1) 用户输入的单行文本 **line**

数据流的起点是用户通过控制台输入的一行文本，**line** 是程序处理数据的最小单元。用户每次在控制台输入一行文本后，**line** 被临时存储，随后传递给 **readInput** 组件进行处理。用户可以通过逐行输入文本的方式提供数据，直到输入关键字“END”来结束输入流程。**line** 的存在使得程序可以灵活地逐步接收用户的输入，为后续的数据处理提供基础保障。作为最基本的连接件，**line** 承载着用户的即时输入，并最终被汇集到字符串向量 **lines** 中，用于后续的移位和排序操作。

(2) 字符串向量 **lines**

lines 是由 **readInput** 组件生成的字符串向量 (`vector<string>`)，其中每个元素表示用户输入的一行文本。该连接件整合了用户输入的所有文本行，将其作为完整的输入数据结构传递给 **generateShifts** 组件。**lines** 的作用是将分散的单行输入组织为一个整体，方便后续处理逻辑的实现。作为程序中的第一个主要数据连接件，**lines** 是文本处理流程的起点，为生成循环移位的逻辑提供了完整的原始数据。

(3) 字符串向量 **shifts**

字符串向量 **shifts** 先由 **generateShifts** 组件生成，其中每个元素表示用户输入的某一行文本的一个循环移位结果。**shifts** 包含了对每一行文本进行所有可能移位后得到的排列组合。这些结果被组织为字符串向量并传递给 **alphabetize** 组件。作为程序的主要数据连接件，**shifts** 是文本处理流程的中间状态，承载了对原始数据的结构化转换，体现了系统的核心处理逻辑。

在 **alphabetize** 组件中，未排序的 **shifts** 被按字母顺序重新排列，生成的结果直接覆盖原始 **shifts** 数据。这些排序后的结果是程序的最终输出内容，被传递到 **writeOutput** 组件以展示给用户。排序后的 **shifts** 是程序中的关键数据连接件，标志着数据处理的完成状态，其作用是为用户提供清晰、有序的文本输出。

在整个程序中，连接件承担了数据在组件之间传递的重要角色。**line** 是用户输入的最小单元，为后续的文本集合提供基础数据；**lines** 是输入的完整集合，为文本处理逻辑提供整体数据；**shifts** 则分为未排序和已排序两种状态，分别用于表示处理结果的中间阶段和最终状态。通过这些连接件，数据得以在系统中高效流动，并按照预定的逻辑顺序完成从输入到输出的全流程处理。这种设计确保了数据传递的清晰性和组件之间的协作性，是程序逻辑得以顺畅运行的关键。

1.2 流程图说明

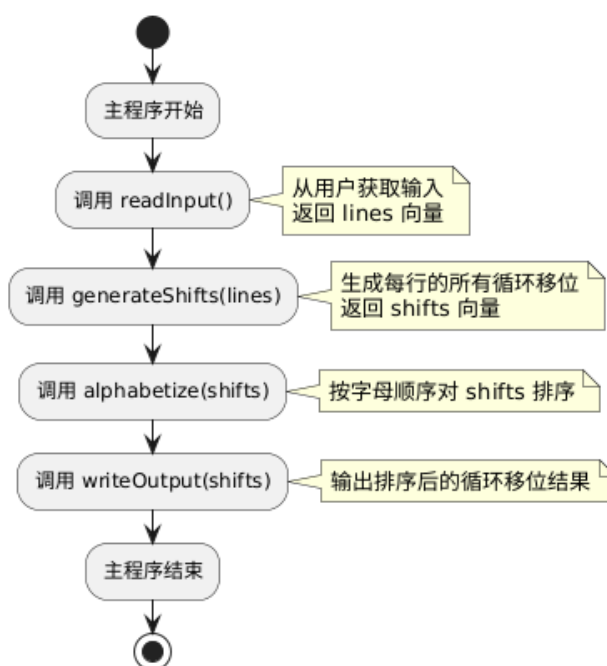


Figure 1.2 主程序子程序风格实现流程图

1.3 结果界面截图

```
Microsoft Visual Studio 调试
主程序子程序风格实现:
请输入行 (输入 'END' 结束):
hello world
that is ok
the only limit is doubts
END

排序后的循环移位结果:
doubts the only limit is
hello world
is doubts the only limit
is ok that
limit is doubts the only
ok that is
only limit is doubts the
that is ok
the only limit is doubts
world hello

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_MainProgramSubroutine.exe (进
程 38420)已退出, 代码为 0。
```

The screenshot shows the output of the program in a terminal window. It displays the input text, the generated shifts, and the sorted shifts. The input text is "hello world", "that is ok", "the only limit is doubts", and "END". The sorted shifts are "doubts the only limit is", "hello world", "is doubts the only limit", "is ok that", "limit is doubts the only", "ok that is", "only limit is doubts the", "that is ok", "the only limit is doubts", and "world hello". The terminal window title is "Microsoft Visual Studio 调试".

Figure 1.3 主程序子程序风格结果截图

1.4 风格优缺点

1.4.1 优点

a. 设计逻辑清晰，易于理解

系统的设计基于主程序-子程序的架构风格，整体逻辑按照数据处理流程从输入到输出依次执行，具有清晰的顺序性和直观的设计思路。每个函数完成一个明确的任务，职责单一且相互独立。例如，`readInput()` 函数专注于从用户输入中逐行读取文本，而 `alphabetize()` 函数只负责对数据进行字母顺序排序。由于每个模块的职责高度聚焦，开发者能够快速理解程序的功能和流程，而无需深入其他模块的实现细节。主程序通过调用这些子程序，清晰地展现了从输入到输出的完整数据处理链条，使得程序的逻辑非常容易把握。

b. 易于调试和维护

每个函数都实现了明确的功能模块，且相互之间通过参数传递数据，这种独立封装的设计使调试和维护变得更加高效。当程序出现问题时，可以通过检查各模块的输入和输出，迅速定位到具体的问题所在。例如，如果排序结果有误，可以直接调试 `alphabetize()` 函数，而无需遍历整个程序逻辑。此外，模块化的设计还方便了测试过程，开发者可以对每个子程序进行独立的单元测试，从而确保各模块功能的正确性。这种清晰的模块化设计有效降低了系统的维护成本，提升了开发效率。

如图 1.4 所示，程序执行过程中展示了循环移位和排序后的结果。从输出可以明显看出，程序的循环移位功能实现良好，每一句输入的文本行都完成了所有可能的循环移位操作。这表明 `generateShifts` 组件运行正常，能够正确生成移位后的字符串。然而，从排序后的结果可以观察到，排序并未按照字典序排列，而是以一种非预期的顺序展示。这表明问题出现在 `alphabetize` 组件中，特别是在排序逻辑或比较函数的实现上。这种模块化的架构风格有效地将问题范围缩小到具体模块，显著提升了程序的可调试性和维护效率，易于调试维护的优点在此得到了充分体现。



```
Microsoft Visual Studio 调试
主程序子程序风格实现(易于维护特点展示):
请输入行 (输入 'END' 结束):
hello world
that is ok
the only limit is doubts
END

排序后的循环移位结果:
world hello
the only limit is doubts
that is ok
only limit is doubts the
ok that is
limit is doubts the only
is ok that
is doubts the only limit
hello world
doubts the only limit is

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_MainProgramSubroutine.exe (进
程 23804)已退出, 代码为 0.
```

Figure 1.4 主程序子程序风格的问题定位示例

c. 模块化、数据共享

程序的设计充分体现了模块化思想，每个功能被封装到独立的函数中，模块之间通过共享

的数据存储进行交互。这种设计使得数据在不同模块之间的传递更加高效，同时保持了模块的独立性。例如，用户输入的数据被存储为一个字符串向量 `lines`，并作为参数传递给生成移位的模块；而移位生成后的结果 `shifts` 则直接传递给排序模块进行处理。通过这种逐步细化的设计，复杂任务被分解为多个独立的模块，从而降低了系统开发的复杂度。

1.4.2 缺点

a. 模块耦合度高，扩展性差

主程序-子程序架构中的模块通过参数紧密耦合，这种设计虽然能够实现数据的顺畅流动，但在功能扩展上存在一定的局限性。在本项目 KWIC 中，如果需要在现有流程中新增一个数据处理模块如数据过滤模块 `filterData` 在移位前对输入的数据进行过滤，开发者必须修改主程序中现有的模块调用顺序，并将新的功能模块嵌入到流程中，新流程图如图 1.5 所示。这种紧密的耦合限制了模块的灵活性，同时增加了维护和扩展的难度。此外，当数据流的复杂性增加时，模块之间的相互依赖可能导致数据传递变得更加复杂，甚至引入潜在的错误。

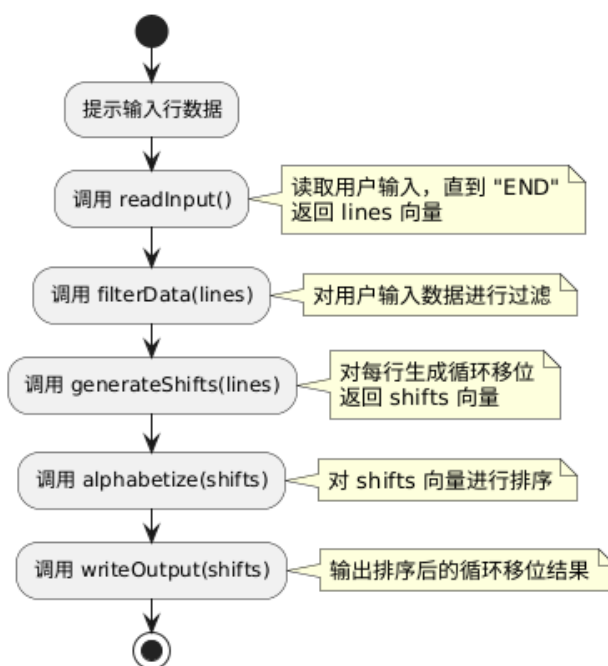


Figure 1.5 KWIC 添加模块后的新流程图

b. 代码复用性低

主程序-子程序架构中的函数通常是特定的数据处理流程设计的，这导致代码的通用性较低。在不同项目中复用现有的模块时，由于依赖于特定的数据格式和处理逻辑，往往需要对代码进行修改或重新设计。在本项目 KWIC 的实现中，`generateShifts` 移位模块对数据的输入和输出有严格的要求，必须输入 `vector<string>` 格式的数据，输出也同样是 `vector<string>` 格式，无法直接应用到其他场景。缺乏通用性的代码设计不仅限制了模块的跨项目复用能力，也增加了开发人员的工作量和系统复杂性。

c. 全局控制集中，灵活性不足

在主程序-子程序架构中，主程序承担着全局控制的责任，负责依次调用子程序并协调数

据流动。然而，这种控制方式是固定的，缺乏动态调整的能力。在本项目 KWIC 的实现中，主程序按照既定的顺序调用 `readInput` 输入、`generateShifts` 移位、`alphabetize` 排序和 `writeOutput` 输出模块，无法根据实际需求灵活调整处理流程。这种固定的控制逻辑在需求变更时表现出较大的局限性，尤其当流程需要动态调整或模块的执行顺序需要改变时，现有架构难以满足灵活性的要求。

d. 难以支持大型系统的维护

随着系统规模的增大和模块数量的增加，主程序-子程序架构的维护成本也会大幅上升。由于模块之间存在紧密的参数耦合关系，当某个模块需要修改时，可能会对其他模块产生连锁反应，增加了维护的难度。此外，随着代码量的增加，逻辑复杂性逐步提升，系统变得难以理解、测试和维护。本项目中，主程序在调用 `readInput()`、`generateShifts()`、`alphabetize()` 和 `writeOutput()` 时，其执行顺序是固定的，如果需要动态调整模块的执行顺序，主程序可能需要大幅修改，从而引入更多潜在的错误。结构化程序在小型项目中具有一定的优势，但在大型系统中难以保持清晰的逻辑结构和较低的维护成本。

1.5 总结

主程序-子程序架构是一种经典的设计方法，适合较小规模和相对稳定的任务，具有设计逻辑清晰、模块化强、易于调试的优点。然而，随着系统规模的扩大和需求的变化，该架构的扩展性和灵活性不足的问题会逐渐显现。由于模块之间的紧密耦合和数据共享，这种架构对数据格式和控制流程的变化非常敏感，增加了维护的难度。因此，该架构更适合于一次性开发的小型项目，或处理线性流程的数据处理任务。而对于需要频繁扩展和调整的复杂系统，应考虑使用更灵活的架构以提高系统的可扩展性和可维护性。

第二章 批处理风格

2.1 批处理风格设计

在本章的 KWIC 项目中，系统采用了批处理风格的设计模式。这种架构将整个程序划分为若干独立的模块，每个模块专注于完成某一特定的任务。模块之间通过定义明确的数据流进行连接，数据从输入端开始，依次流经各模块，逐步完成从输入到输出的整个处理流程。这种设计严格限制了模块间的数据共享，所有交互均通过管道形式的数据流完成，从而实现模块高内聚和系统低耦合的特性。

在批处理架构中，每个模块是独立的且职责明确。在接收到数据时，模块能够自主启动处理逻辑，并将结果传递给下一个模块。各模块之间无需了解彼此的实现细节，只需遵循统一的数据接口。这种模块化的设计方式极大地提升了系统的灵活性与可扩展性。例如，当需要替换某个模块时，只需对该模块进行修改，而无需调整其他模块或改变系统整体结构。

在本程序中，整个数据处理流程被分为四个模块：从用户读取输入 `inputFilter`、将输入行生成所有可能的循环移位 `shiftFilter`、对生成的循环移位进行排序 `alphabetizeFilter`、输出排序后的循环移位结果 `outputFilter`，逐步完成从数据输入到结果输出的完整处理过程。各模块之间的协作通过共享的数据存储实现，数据依次从一个模块流向下一个模块，减少了模块间的复杂依赖，确保了数据处理的有序性和系统的高性能。这种架构不仅体现了模块高内聚低耦合的设计理念，同时也提供了直观的实现方式，批处理风格实现 KWIC 设计图如图 2.1 所示，展示了各个模块之间的交互关系及数据流动方式，为理解 KWIC 系统的实现提供了清晰的参考。

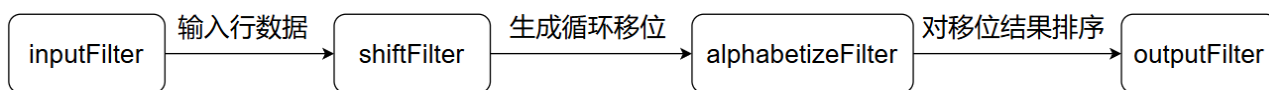


Figure 2.1 批处理风格实现 KWIC 设计图

2.1.1 组件

在批处理风格的实现中，程序被分解为四个主要的模块。每个模块独立完成特定任务，分别负责数据的输入、循环移位、排序以及输出。通过这种模块化的设计，系统实现了高内聚和低耦合，各组件之间通过明确的数据流进行交互，彼此独立，便于维护和扩展。

(1) 读取用户输入 `inputFilter`

该组件作为数据流的起点，负责从用户读取输入文本，直到用户输入关键字“END”表示结束。用户输入的每一行文本被处理为一个字符串 `line`，并依次存储在一个字符串向

量 `lines` 中。通过这一组件，程序实现了从非结构化输入到结构化数据的转换，为后续组件提供标准化的输入格式。`inputFilter` 仅专注于输入数据的处理，与其他组件无直接耦合，体现了单一职责的设计原则。

(2) 生成循环移位 `shiftFilter`

`shiftFilter` 组件接收来自 `inputFilter` 的字符串向量 `lines`，针对每一行文本生成所有可能的循环移位。它对每一行的单词顺序重新排列，生成的移位结果被存储为新的字符串向量 `shifts`。该组件完成了对输入数据的结构化转换，也确保了每一行输入文本都能够被充分分析和处理。`shiftFilter` 的设计逻辑清晰，数据处理与传递独立封装，确保了模块的高可复用性。

(3) 对移位结果排序 `alphabetizeFilter`

`alphabetizeFilter` 接收未排序的循环移位数据 `shifts`，对其进行字母顺序的排序。排序完成后，结果被存储为新的有序字符串向量 `sortedShifts`，并传递给 `outputFilter` 输出模块。该组件在数据流中起到了规范化的作用，是连接移位生成与输出展示的重要环节。

(4) 输出结果 `outputFilter`

`outputFilter` 是程序的数据输出终端，接收来自 `alphabetizeFilter` 的排序结果 `sortedShifts`，并将其逐行打印到控制台，供用户查看。作为最后一个模块，它专注于将数据展示给用户，完成了从输入到输出的闭环处理流程。`outputFilter` 的职责明确，仅负责输出逻辑，与其他模块保持独立。

以上四个组件以模块化的方式协同工作，完成了程序从输入到输出的整个数据处理流程。每个组件独立处理其自身逻辑，数据通过连接件在组件之间流动。模块化设计不仅提高了系统的可维护性和可扩展性，也确保了功能职责的单一性和逻辑清晰性。

2.1.2 连接件

在批处理风格中，连接件是组件之间的数据传递桥梁，负责确保数据按照固定的顺序在组件间依次流动。每个连接件将上一个组件的输出作为下一个组件的输入，确保了整个数据处理流程的连续性和一致性。通过明确的数据结构，连接件在程序中建立了清晰的逻辑关系。

(1) 用户输入的单行文本 `line`

用户通过控制台逐行输入文本，每一行作为独立的字符串传递给 `inputFilter` 组件。该连接件是数据流的起点，承载了原始的用户输入，确保程序能够接收多样化的输入数据。在 KWIC 系统中，这些单行文本是系统数据流的初始状态，其内容未经任何处理，直接反映了用户的输入意图。通过这种逐行输入的方式，程序可以灵活应对用户提供的不同内容，无论是短文本、长段落，还是包含多种字符的复杂输入，都能够以标准化的形式开始数据处理流程。

(2) 用户输入的字符串向量 `lines`

`inputFilter` 组件将用户输入的每一行文本存储为一个字符串向量，其中每个元素表示用户输入的一行。这一连接件负责将结构化的输入数据传递给 `shiftFilter`，为后续的移位处理提供了清晰、标准化的基础数据格式。通过将用户输入转化为字符串向量，系统

能够更高效地对数据进行批量处理，而不需要逐行操作。这种设计确保了移位处理模块能够专注于数据重组的逻辑，而无需关心输入数据的格式问题。

(3) 循环移位生成的字符串向量 `shifts`

`shiftFilter` 组件接收字符串向量 `lines` 作为输入，并对每一行生成所有可能的循环移位。生成的移位结果被组织为新的字符串向量 `shifts`，其中每个元素表示一种可能的排列组合。这一连接件是 KWIC 系统数据流的重要中间状态，不仅承载了循环移位的处理结果，还为排序模块提供了多样性和丰富性的输入数据。通过这一连接件，移位数据的重组结果能够无缝传递给下游的排序模块，为最终结果的规范化提供了完整的数据基础。

(4) 排序后的字符串向量 `sortedShifts`

`alphabetizeFilter` 组件接收未排序的循环移位结果，对其进行字母顺序排序，并输出一个经过排序的字符串向量 `sortedShifts`。该连接件是数据流的终点，承载了完整处理的规范化结果，确保最终输出能够符合用户的期望。通过 `sortedShifts`，排序后的数据以一种清晰、用户友好的顺序传递给 `outputFilter` 模块，便于输出模块将数据展示给用户。这一连接件的设计使得数据流从最初的混乱状态逐渐变得有序，为系统的最终输出奠定了规范化基础。

连接件在批处理架构中起到了至关重要的作用，负责数据的传递与流转。通过明确的连接件设计，程序的逻辑更加清晰，组件间的交互更加有序。每个连接件都以标准化的数据格式为基础，为模块的独立性和系统的扩展性提供了保障。

2.2 流程图说明

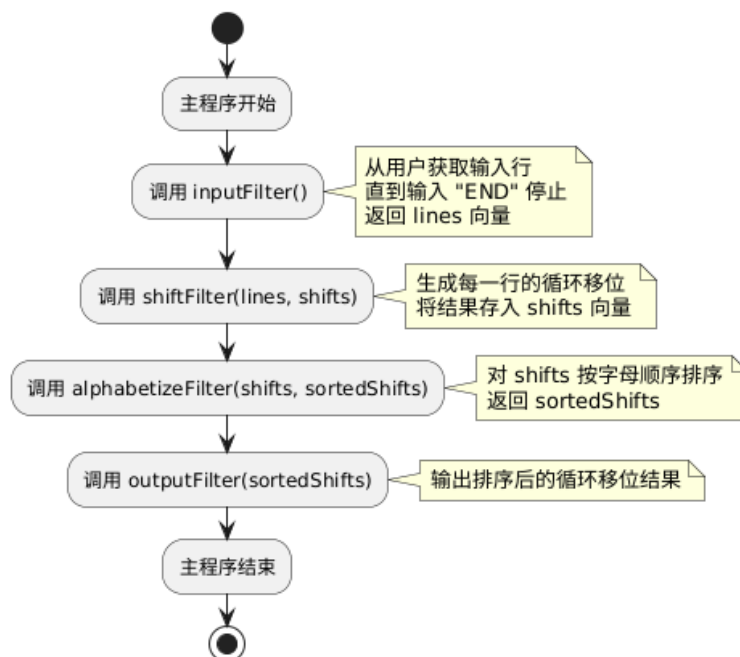
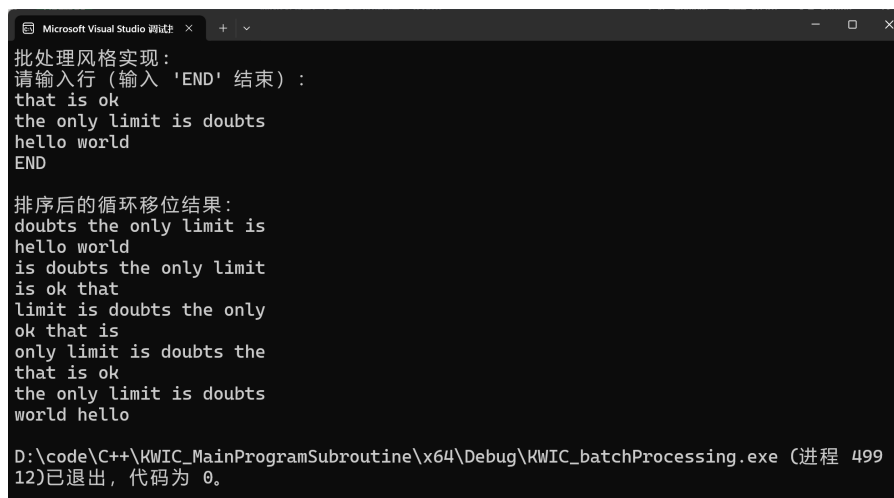


Figure 2.2 批处理风格 KWIC 实现流程图

2.3 结果界面截图



```
Microsoft Visual Studio 调试  x + -
批处理风格实现：
请输入行（输入 'END' 结束）：
that is ok
the only limit is doubts
hello world
END

排序后的循环移位结果：
doubts the only limit is
hello world
is doubts the only limit
is ok that
limit is doubts the only
ok that is
only limit is doubts the
that is ok
the only limit is doubts
world hello

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_batchProcessing.exe (进程 49912)已退出，代码为 0。
```

Figure 2.3 批处理风格结果截图

2.4 风格优缺点

2.4.1 优点

1) 过程流非常直观

在批处理架构中，各模块按照数据流的顺序依次处理输入，并将结果传递到下一个模块。每个模块专注于完成其特定功能，整个流程从用户输入到结果输出非常清晰且直观。对于当前实现的 KWIC 系统，从输入读取入 `inputFilter` 开始，经过循环移位 `shiftFilter`、排序 `alphabetizeFilter`，再到输出 `outputFilter`，所有步骤都按照固定的逻辑依次执行，逻辑上层层推进，便于开发者理解和维护。

2) 高模块化，支持复用和易于修改

每个模块专注于单一功能，相互隔离，具有高度模块化的特性。这种设计使得系统能够保持清晰的逻辑结构，各模块的实现互不依赖，彼此间仅通过明确的数据流进行交互。这种模块化设计提供了多个显著的优势，特别是在测试、复用方面表现尤为突出。

在 KWIC 系统中，各模块按照功能划分为输入、移位、排序和输出四个部分，每个部分的功能独立且职责单一。例如，`shiftFilter` 模块专注于生成输入文本的所有循环移位，而 `alphabetizeFilter` 模块则只负责对循环移位结果进行排序。这种设计使得每个模块可以单独进行测试，无需依赖其他模块的实现逻辑或运行状态。例如，开发者可以单独测试 `shiftFilter` 的移位逻辑，验证其是否正确生成所有可能的排列，而不需要关心排序和输出部分的实现。模块的高度独立性不仅降低了测试的复杂性，还提高了开发效率。

此外，模块化设计极大地提高了系统的复用性。在 KWIC 系统中，每个模块可以作为独立的功能单元，方便地移植到其他项目中。例如，`alphabetizeFilter` 模块是一个通用的排序组件，可以直接应用于其他需要字母顺序排序的场景，而无需做额外修改。同

样，`shiftFilter` 模块的循环移位逻辑也具有通用性，可以在任何需要文本排列组合的程序中单独使用。

3) 系统具有并发性

各模块之间相互独立，理论上可以并行处理。这种并行化的特性对于处理大规模数据流的应用场景尤为重要，例如处理成千上万行文本的循环移位和排序任务时，可以显著提升系统性能。

例如，在当前 KWIC 系统中，可以将循环移位的生成与排序的执行进行并发处理，以提高系统的效率。循环移位模块可以逐行生成位移结果，而排序模块在接收到部分结果后，可以立即开始部分排序或临时存储，而无需等待循环移位完全结束。通过并发执行，循环移位和排序的时间复杂度可以部分重叠，进而减少系统的总运行时间。

4) 功能扩展性强

由于模块之间的独立性，新增功能变得非常简单。通过在现有模块中插入新的模块即可实现功能扩展，而无需对整体系统结构进行调整。这种插入式的扩展方式充分体现了批处理架构的灵活性，无论扩展的功能有多复杂，现有的系统结构和逻辑都无需发生变化。

例如，若需对移位后结果进行数据过滤，只需添加一个数据过滤的模块，而不需要对其他过滤器做任何更改，新增数据过滤模块后新的设计图如图2.4所示，这种设计使得系统的功能扩展变得快速且高效。



Figure 2.4 新增数据过滤后的设计图

2.4.2 缺点

1) 无法支持交互式系统

批处理风格更适合用于处理流式数据，但在需要频繁用户交互的系统中表现出较大的局限性。在当前实现的 KWIC 系统中，程序采用批处理模式，用户必须一次性输入所有数据，待处理完成后才能获得最终结果。这种设计无法实时响应用户的动态输入和调整，导致交互灵活性不足，对于需要即时反馈的高交互性应用不够适用。因此，为满足实时交互需求，可以考虑采用事件驱动或观察者模式等更加灵活的架构，以实现输入与输出的动态联动，提升系统的响应能力和用户体验，而不是使用批处理风格来实现。

2) 缺乏通用的数据传输标准

在本系统中，过滤器之间的数据传递依赖于特定的数据结构（如 `vector<string>`），缺乏统一的传输标准。虽然这种设计在小型程序中较为简便，但当系统规模扩大时，数据结构的不一致可能导致集成和维护变得困难。例如，如果某模块的输入和输出数据格式发生改变，可能需要同步调整多个模块，增加了维护成本。

在 KWIC 的实际实现中，每个模块的输入和输出均基于 `vector<string>` 这一数据结构。例如，`inputFilter` 模块接收用户逐行输入的文本，将其存储为字符串向量 `lines`，然后传递

给 `shiftFilter`。`shiftFilter` 接收 `lines` 后，生成包含所有可能循环移位的字符串向量 `shifts`，再将其传递给 `alphabetizeFilter` 进行排序。最后，`outputFilter` 模块接收排序后的字符串向量 `sortedShifts`，将结果逐行输出到控制台。这种数据结构的一致性在当前实现中非常有效，但其依赖性也带来了隐患。

如果某个模块的输入或输出数据结构发生变化，就可能对整个系统产生连锁反应。例如，如果需要对 `shiftFilter` 的输出结果进行扩展，新增移位文本的起始位置作为附加信息，此时 `shifts` 的数据结构可能需要从 `vector<string>` 修改为 `vector<pair<string, int>>`。然而，由于 `alphabetizeFilter` 模块依赖 `shifts` 的原始格式，这种修改会导致排序逻辑无法正常运行，进而需要同步调整排序模块的实现。这种连锁反应使得简单的功能扩展变得复杂，不仅增加了开发成本，也可能引入新的错误。

3) 难以进行错误处理和调试

由于数据流经多个独立的模块，错误的定位和处理可能需要耗费较多时间。例如，若排序结果有误，开发者需要逐一验证每个模块的输入和输出，才能找到问题的根源。这种逐层检查的调试模式，在处理复杂数据流或大规模输入时尤其耗时，可能影响开发效率。

例如在 KWIC 中，如果最终输出结果出现错误，首先需要检查 `shiftFilter` 的输出 `shifts`，以确认是否正确生成了所有可能的循环移位；然后需要检查 `alphabetizeFilter` 的输入 `shifts` 是否与 `shiftFilter` 的输出一致，进一步验证排序逻辑是否按照预期执行；最后，还需要检查 `outputFilter` 是否正确接收并打印了排序后的结果。

每个模块的独立性虽然有助于提高系统的灵活性，但在调试中却增加了复杂性。因为任何一个模块的错误都会影响后续模块的行为。例如，如果 `shiftFilter` 生成的移位结果存在数据丢失问题，排序模块可能表现出错误的行为，但问题的实际根源在 `shiftFilter` 中。这种逐层检查的调试模式，在处理复杂数据流或大规模输入时尤其耗时，不仅降低了开发效率，还增加了调试的难度。

4) 空间复杂性高，系统性能受限

在当前实现中，模块之间通过复制数据向量来传递信息。这种数据复制方式在处理大规模文本数据时，会显著增加内存占用。例如，在循环移位生成和排序之间，每次传递的字符串向量都需要完整复制，导致内存消耗增加。尽管批处理架构支持并发处理，但数据复制的开销限制了系统的性能提升。

比如 `shiftFilter` 接收 `lines`，生成大量循环移位并存储在 `shifts` 中，对于一行包含多个单词的文本，其可能的移位组合数量会随单词数量呈线性增长；`alphabetizeFilter` 从 `shifts` 中接收数据并对其进行排序，这需要将 `shifts` 的内容复制到新的 `sortedShifts` 中；这种数据复制方式会导致内存占用的显著增加。

5) 局限性较大，扩展性能有限

批处理架构中，模块间的独立性是其核心特性，这一设计提高了系统的灵活性，但也带来了明显的扩展局限性。当前 KWIC 系统中，各模块处理的数据是完全独立的，通过固定的数据流进行交互。

比如 `shiftFilter` 仅负责生成循环移位，`alphabetizeFilter` 仅依赖于 `shiftFilter` 的输出进行排序。如果需要在排序时引入与移位过程相关的上下文信息（例如移位的起始位置或行

号), 当前的架构无法直接支持, 除非在 `shiftFilter` 的输出数据中显式添加上下文信息。这种模块化的设计虽然简化了每个模块的逻辑, 但在需要共享状态或依赖全局上下文的场景中显得力不从心。例如, 如果需要通过一种全局优化的排序逻辑, 需要综合考虑所有行的移位结果, 此时各模块之间的独立性反而会成为扩展的障碍, 因为上下文信息无法在模块间高效共享。

此外, 当新增功能需要跨模块数据协作时, 也会面临设计上的挑战。例如, 在当前架构中, 如果需要统计排序后移位结果的某些特定属性 (如重复次数或长度分布), 就必须在多个模块间引入新的数据流或共享机制, 这可能破坏系统的模块独立性, 增加设计和实现的复杂性。

2.5 总结

通过采用批处理架构, 该系统实现了高度模块化和良好的可扩展性。每个模块专注于单一任务, 如输入读取、循环移位生成、排序和输出, 从而确保各个模块的独立性。这种设计不仅提升系统的复用性, 还使得新功能的集成更加灵活。在需要扩展功能时, 可以轻松地在现有模块中添加新的模块, 而无需大幅修改现有模块, 从而显著降低维护成本。

然而, 尽管批处理架构在模块化和可扩展性方面表现出色, 当处理大规模数据集时, 系统可能会面临性能瓶颈和内存开销的挑战。由于各个模块之间通过复制数据来传递信息, 在处理大量文本或数据时, 这种数据复制可能会导致内存占用增加, 进而影响系统的整体性能。

尽管如此, 这种架构风格仍然非常适合数据流处理的应用场景, 特别是在涉及文本处理、数据转换以及数据分析的系统中。其清晰的处理流程和模块划分, 使得数据处理的每个阶段都易于理解和调试。因此, 在需要将输入数据按流水线方式逐步处理的应用中, 批处理架构依然是一个高效且灵活的选择, 尤其在处理结构化数据或流水线任务时, 能够充分发挥其优势。

第三章 面向对象风格

3.1 面向对象风格设计

本章 KWIC 项目采用面向对象（Object-Oriented）设计风格，通过将数据与作用在数据上的操作封装为独立的对象，并利用主控制对象（即主函数 `main`）协调这些对象之间的调用。这种设计方式有效地实现了模块化，将功能职责分离，提高了代码的可维护性和扩展性。

在程序中，数据和操作被封装为独立的类对象，包括 `Input`、`CircularShift`、`Alphabetizer` 和 `Output`。每个类对象专注于单一的功能，并通过提供封装的接口与主控制对象进行交互。例如，`Input` 类封装了从用户输入读取文本的操作，而 `CircularShift` 类专注于生成输入文本的所有可能循环移位组合，`Alphabetizer` 类负责对循环移位结果按照字母顺序进行排序，`Output` 类则负责将处理结果输出到用户界面。通过这种封装，程序的每个模块可以独立开发、测试和复用，从而减少模块之间的依赖性。

主程序（`main` 函数）作为主控制对象，负责协调各个类对象之间的操作顺序。数据从 `Input` 类读取后，依次流经 `CircularShift` 类、`Alphabetizer` 类，最终通过 `Output` 类输出结果。主程序的职责是确保数据按照既定流程依次传递，各模块专注于其自身的功能逻辑，避免了模块之间的直接依赖，从而保持了系统的高内聚低耦合特性。

面向对象的设计架构清晰地体现在 KWIC 系统的设计图中，如图 3.1 所示。图中展示了各类对象之间的交互关系，以及主程序对模块的整体协调。通过该设计图可以直观地看到，程序中每个模块的功能划分和执行流程。该设计图不仅直观地展现了 KWIC 系统的功能模块划分，还反映了面向对象设计风格的核心思想——通过对象封装数据和操作，通过主控制对象协调流程。通过这种设计，程序在功能扩展、逻辑修改等方面具有较高的灵活性。

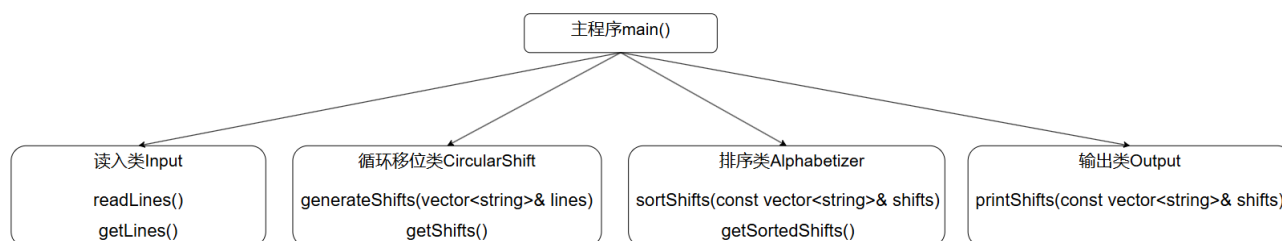


Figure 3.1 面向对象风格实现 KWIC 设计图

3.1.1 组件

在 KWIC 项目中，组件是面向对象设计的核心部分。程序通过将数据处理的不同职责封装到独立的类中，形成了模块化的设计架构。这些组件通过清晰的接口互相协作，构建了一个高效的 KWIC 系统。

(1) Input 组件

Input 组件负责从用户输入中读取数据。它通过提供一个接口方法逐行获取用户输入，直到用户输入关键字 "END" 为止。所有输入内容被存储为一个字符串向量 `lines`，其中每一行文本 `line` 作为一个独立元素。这一组件的核心职责是将非结构化的用户输入转化为结构化的数据，为后续处理提供一致的格式化基础。通过这种封装方式，输入的逻辑被隔离，便于修改或扩展输入源。

(2) CircularShift 组件

CircularShift 组件接收 Input 提供的字符串向量 `lines`，并生成每一行文本的所有可能的循环移位。其核心逻辑通过遍历单词的顺序重新排列，确保生成的移位涵盖所有可能组合。移位结果存储为新的字符串向量 `shifts`，供排序模块使用。该组件的设计重点在于数据的变换逻辑，其独立性保证了移位算法可以在不同场景中复用，同时其输出格式规范，为后续模块提供了高质量的数据。

(3) Alphabetizer 组件

Alphabetizer 组件的功能是对 CircularShift 生成的循环移位结果 `shifts` 进行排序。排序逻辑按字母顺序进行。排序后的移位存储为新的字符串向量 `sortedShifts`，并传递给输出模块。这一组件的独立性确保了排序逻辑可以被调整或替换，例如支持其他排序规则（如逆序排序或特定字符优先排序），从而提高了系统的灵活性。

(4) Output 组件

Output 组件是系统的数据展示模块。它接收 Alphabetizer 提供的排序结果 `sortedShifts`，并逐行输出到控制台。输出的内容以规范的顺序显示，便于用户直观地查看程序的处理结果。作为整个数据流的终点，Output 的职责是将程序的最终结果展示给用户。

通过上述组件的划分，程序实现了输入、移位、排序和输出的职责分离。每个组件专注于完成特定的任务，模块之间通过明确的接口协作，形成了一种松耦合、高内聚的设计模式。

3.1.2 连接件

连接件是组件之间数据交互的桥梁，承载了程序中数据流动的核心逻辑。在 KWIC 系统中，连接件通过明确的数据结构（如字符串向量）实现了各模块之间的无缝衔接。

(1) 用户输入的字符串向量 (`lines`)

`lines` 是 Input 组件输出的结果，其中每个元素是用户输入的一行文本 `line`。该连接件负责将用户输入的非结构化数据转化为结构化的形式，作为输入传递给 CircularShift。这一数据结构的稳定性为移位模块的处理提供了坚实基础，同时确保了数据流的规范化。

(2) 循环移位生成的字符串向量 (`shifts`)

`shifts` 是由 CircularShift 生成的结果，包含所有循环移位的文本。这一连接件是数据流的中间状态，承载了移位逻辑的输出，为排序模块提供了丰富的输入数据。这一数据结构反映了移位模块的处理逻辑，并在排序前保留了所有的排列可能性。

(3) 排序后的字符串向量 (`sortedShifts`)

`sortedShifts` 是 Alphabetizer 的输出结果，包含按字母顺序排列的移位文本。这一连接

件将规范化的排序结果传递给 **Output**，确保最终输出的内容有序且易于用户理解。这一连接件标志着数据流从处理阶段进入输出阶段，是数据最终展示前的最后一个中间状态。

通过连接件的有效设计，程序实现了组件之间的逻辑独立性和数据传递的连贯性。各模块仅通过约定好的数据结构进行交互，从而保持了模块间的松耦合特性，同时为系统的可扩展性提供了基础支持。

3.2 类图说明



Figure 3.2 面向对象风格 KWIC 实现类图

3.3 结果界面截图

```
Microsoft Visual Studio 调试  x + -
面向对象风格实现：
请输入行（输入 'END' 结束）：
the only limit is doubts
hello world
that is ok
END

排序后的循环移位结果：
doubts the only limit is
hello world
is doubts the only limit
is ok that
limit is doubts the only
ok that is
only limit is doubts the
that is ok
the only limit is doubts
world hello

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_ObjectOriented.exe (进程 3043
2)已退出，代码为 0。
```

Figure 3.3 面向对象风格结果截图

3.4 风格优缺点

本程序采用了面向对象设计风格，通过封装数据和操作，协调不同模块之间的交互，形成了模块化的系统架构。以下从优点和缺点两个方面结合实际代码进行详细分析。

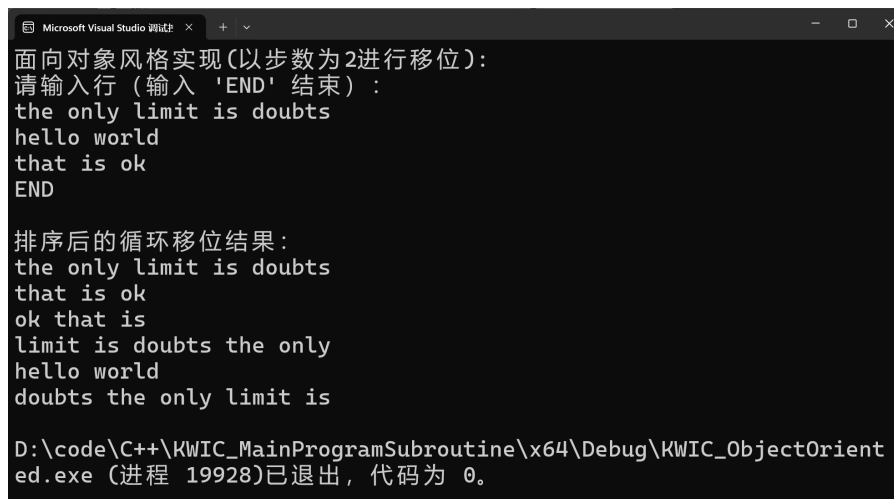
3.4.1 优点

1) 高封装性，低耦合度：

面向对象设计将数据和操作封装在类中，每个类对外只通过接口访问其内部数据。例如，CircularShift 类的内部移位逻辑对其他类是完全透明的，外部只需要调用其 getShifts() 方法获取结果，而无需关心移位的具体实现。这样的封装保证了模块之间的低耦合性，任何一个类的修改都不会影响其他模块的功能，从而提高了系统的灵活性和可维护性。

在实际应用中，若需调整循环移位的逻辑，例如采用步长为 2 的方式进行移位，只需修改 CircularShift 的内部实现，而不需要改动其他模块的代码。这种设计不仅避免了对全局系统逻辑的干扰，还能通过增加参数控制实现不同的功能。具体地说，步长为 2 的移位逻辑只生成部分移位结果，减少了冗余计算和内存占用，从而优化了性能。

图 3.4 展示了采用步长为 2 的移位实现结果。在用户输入文本后，系统生成并排序了移位结果。可以看出，移位逻辑调整为步长为 2 后，仅生成了步长为 2 的移位结果而不是原来步长为 1 的移位结果，同时排序结果仍然保持规范化，符合系统设计的目标。



```
Microsoft Visual Studio 调试  x + -
面向对象风格实现(以步数为2进行移位):
请输入行 (输入 'END' 结束):
the only limit is doubts
hello world
that is ok
END

排序后的循环移位结果:
the only limit is doubts
that is ok
ok that is
limit is doubts the only
hello world
doubts the only limit is

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_ObjectOrient
ed.exe (进程 19928)已退出, 代码为 0.
```

Figure 3.4 面向对象风格实现步长为 2 的循环移位结果

2) 易于扩展和维护：

程序具有良好的扩展性，通过继承和多态可以轻松实现功能扩展。例如，当前系统使用 Alphabetizer 类对循环移位结果按字母顺序排序。如果需要添加自定义排序逻辑，可以通过继承 Alphabetizer 类实现新的排序算法，而不需要修改现有代码。这种设计使得系统能够以较低的成本应对需求变更，同时保证原有功能的稳定性。

3) 高代码复用性：

各个类独立完成特定任务，设计通用且易于复用。例如，CircularShift 类生成循环移位

的功能可以直接应用于其他项目中，只需提供新的输入数据。类似地，`Alphabetizer` 类的排序功能也可以在其他需要排序的场景中使用。这种高复用性不仅减少了代码重复，也降低了开发和维护的成本。

4) 数据隐藏与安全性:

每个类通过封装隐藏了其内部数据，外部模块无法直接访问或修改。以 `Input` 类为例，其存储的用户输入数据仅能通过 `getLines()` 方法获取，而无法被其他模块直接操作。这种设计避免了数据的意外篡改，提高了系统的安全性和可靠性。同时，数据隐藏使得模块的使用更加简单，开发者只需关注接口即可调用功能。

3.4.2 缺点

1) 不适合频繁扩展系统功能:

尽管面向对象设计支持模块化，但当系统需要增加跨模块的新功能时，可能需要对多个类进行修改。这种设计在功能扩展的初期可能表现出较高的灵活性，但随着系统规模的增长和功能的复杂化，其局限性逐渐显现。例如，在本系统中，若需要在排序前对循环移位结果进行过滤，就需要增加一个新的过滤模块 `Filter`，并修改 `CircularShift` 和 `Alphabetizer` 之间的调用逻辑。图 3.5 展示了修改后的类图结构，新增的 `Filter` 模块作为数据处理流程的中间环节，在 `CircularShift` 和 `Alphabetizer` 之间插入。这不仅增加了系统的复杂性，还可能使模块之间的依赖关系变得更加紧密，从而削弱系统的灵活性。此外，在维护和测试时，开发者需要检查新增模块是否与其他模块正确协作，进一步加大了系统维护的复杂性。

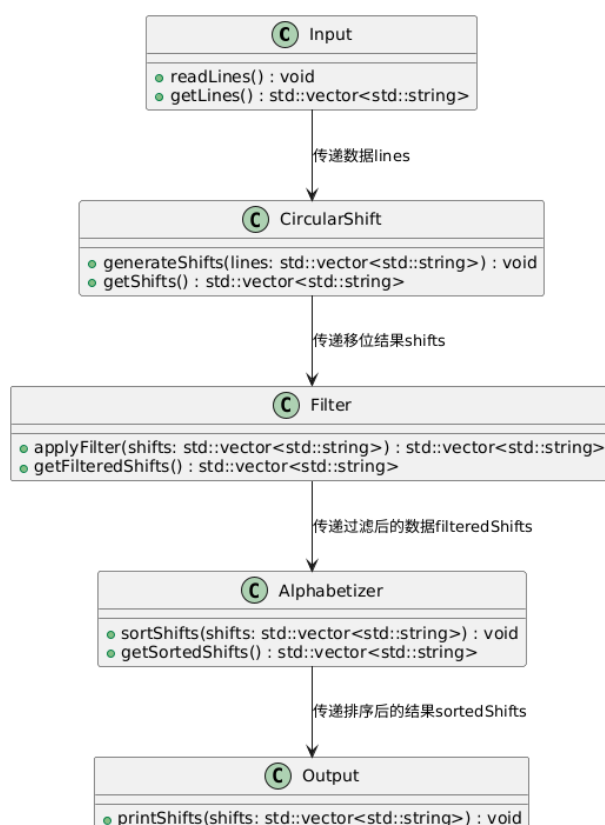


Figure 3.5 面向对象风格扩展数据过滤模块

虽然面向对象设计通过模块化封装提高了代码的复用性和清晰性，但在需要频繁扩展系统功能时，模块之间的依赖关系和调用逻辑可能发生改变，导致维护成本增加。为应对这一问题，可以在设计初期增加接口层或使用松耦合的设计模式（如策略模式），以减少模块间的依赖强度，提升系统的扩展性。

2) 管理大量对象和交互:

在面向对象设计中，程序通过将不同功能分配到多个独立的类中，每个类负责完成特定的任务，并通过方法调用进行交互。然而，当系统的功能变得复杂时，这种设计可能带来对象管理的挑战。在当前系统中，主函数需要协调 `Input`、`CircularShift`、`Alphabetizer` 和 `Output` 等对象之间的交互。虽然这种设计确保了各类功能的模块化，但主函数需要显式地管理所有模块的调用顺序和数据传递逻辑。

如果进一步增加模块数量，例如添加一个数据过滤模块 `dataFilter` 或其他数据处理模块，主函数的逻辑将变得更加复杂。开发者需要仔细调整每个模块的调用顺序，并确保数据在模块之间正确传递。这不仅增加了代码维护的难度，还可能引入新的错误。此外，随着对象数量的增长，开发者需要花费更多精力跟踪对象间的交互和依赖关系，特别是在需要跨模块协调功能时，复杂性会进一步放大。

3) 分布式职责导致复杂性:

面向对象设计通过将行为分散到不同的类中实现了职责分离，这种设计虽然提高了模块的独立性，但也可能导致系统整体逻辑不够直观。例如，在当前系统中，`CircularShift` 类生成的移位数据被传递给 `Alphabetizer` 进行排序，然而这种依赖关系在代码中是通过数据流隐式表达的，并没有显式地说明两者之间的紧密联系。开发者在调试或维护代码时，需要深入理解每个类的实现细节，才能清晰地掌握模块之间的交互逻辑。

尤其是在模块之间存在多个依赖关系时，分布式职责的设计可能增加调试和维护的复杂性。例如，若新增一个模块对数据进行额外处理（如过滤或转换），开发者需要重新分析数据流的路径，并调整模块间的交互方式。这种分布式设计虽然有助于降低单个模块的复杂度，但整体系统的复杂性可能会显著增加，影响开发效率。

4) 性能开销和内存占用:

面向对象设计在对象的创建和方法调用上会产生一定的性能开销。在当前系统中，`CircularShift` 类需要遍历每一行输入数据，并生成所有可能的循环移位，这一过程中会频繁地对输入数据进行复制和重新排列。由于每次生成的移位结果都需要存储到一个字符串向量中，内存占用随着输入数据的增加而迅速增长。此外，在 `Alphabetizer` 模块对数据进行排序时，也需要对移位结果进行新的排序操作，这进一步增加了内存和计算的开销。

这种设计对于小规模数据处理来说问题不大，但在处理大规模数据时，性能和内存消耗可能成为瓶颈。例如，如果输入数据包含数千行文本，每一行的循环移位结果将占用大量内存，同时排序操作的复杂度也会显著增加。尽管面向对象设计提供了清晰的模块化结构，但其对性能的影响需要在设计阶段进行优化，例如通过减少数据复制或使用惰性计算（`lazy evaluation`）来降低内存开销。

综上所述，本程序的面向对象设计风格在模块化、复用性和扩展性方面具有显著优势，

但在频繁功能扩展、复杂对象管理和性能优化方面也面临一定的挑战。在实际应用中，需要根据需求权衡这些优缺点，以设计出最适合的系统架构。

3.5 总结

本程序通过面向对象设计，成功实现了 KWIC (Key Word In Context) 功能的模块化，使各个功能模块独立且可重用。在此设计下，每个模块（如输入处理、循环移位、排序和输出）被封装为独立的类，各模块通过明确的接口进行交互，而不直接共享数据。这种模块化的设计思路带来了高度的灵活性和良好的代码复用性，使得系统在不同场景下更易于扩展和调整。

灵活性体现在系统能够轻松地添加新功能或更换现有算法，而不影响其他模块。例如，如果需要调整排序逻辑，只需替换 `Alphabetizer` 类中的排序方法，而无需修改整个系统的结构。此外，由于各模块彼此独立，开发人员可以针对单个模块进行独立测试和调试，从而提高开发效率和系统的可维护性。

然而，尽管该架构具备很高的灵活性和复用性，在应对复杂功能扩展时也面临一些挑战。随着系统规模的扩大，对象之间的依赖关系和交互复杂度可能会迅速增加，从而导致系统维护变得更加困难。当需要在现有的处理流程中引入新功能时，可能会涉及对多个模块的调整，从而增加维护和扩展的复杂性。

因此，该架构更适用于功能相对稳定、模块化处理和数据封装要求较高的应用场景。在这些场景中，模块职责分明且变化较少，能够充分发挥面向对象设计的优势，实现系统的可扩展性和代码复用性。对于需要频繁调整功能的系统，可能需要更灵活的架构或更轻量的设计模式，以减少模块之间的耦合度，提高整体的开发和维护效率。

第四章 黑板风格

4.1 黑板风格设计

本章的 KWIC 项目采用了黑板（Blackboard）架构设计，这种设计通过共享的黑板作为系统核心存储区域，整合了多模块协作解决问题的机制。黑板架构广泛用于需要松散耦合组件协作的系统，通过集中式数据管理和动态模块控制，实现了功能模块化和高效数据共享。系统由黑板数据结构、知识源模块和控制器三大核心部分组成，各部分协同工作以完成 KWIC 系统的功能。

在这种设计中，黑板在系统中充当全局共享的存储区域，集中存储了整个系统所需的数据，并以统一的方式为各模块提供访问和更新的接口。它保存了用户输入的文本、循环移位结果以及排序后的最终结果等核心数据。在系统运行中，模块从黑板中读取当前状态的数据，执行特定的操作后，将更新结果重新写回黑板。通过这种方式，模块之间完全解耦，不需要直接通信，从而降低了系统的复杂性。

黑板架构的功能通过多个知识源模块实现，每个知识源模块专注于解决系统中的某一部分问题。在 KWIC 系统中，这些模块包括输入模块（Input）、循环移位模块（CircularShift）、排序模块（Alphabetizer）和输出模块（Output）。Input 模块负责读取用户输入，将逐行输入的文本写入黑板的 `lines` 字段，为后续处理提供结构化的原始数据。接下来，CircularShift 模块读取黑板中的 `lines` 数据，生成每一行文本的所有可能循环移位组合，并将结果写入 `shifts` 字段。随后，Alphabetizer 模块从黑板中获取未排序的移位结果 `shifts`，对其按照字母顺序进行排序，并将排序后的结果存入 `sortedShifts` 字段。最后，Output 模块读取 `sortedShifts` 数据，将最终的排序结果输出到控制台供用户查看。

知识源模块的独立性使得它们能够根据需求单独进行调整和扩展，例如更改循环移位逻辑或添加新的排序算法，而不会影响其他模块的功能。这种设计显著提高了系统的灵活性和可维护性。

为了协调这些知识源模块的执行，黑板架构引入了控制器作为核心的协调机制。控制器负责监视黑板的当前状态，并根据状态变化动态激活相关模块。在 KWIC 系统中，控制器通过主程序（`main` 函数）实现，其主要职责是按照既定的逻辑顺序依次调用输入、移位、排序和输出模块，确保数据流的完整性和正确性。控制器通过监视黑板的状态触发知识源模块的执行。例如，当 `lines` 数据更新后，控制器会激活 CircularShift 模块生成移位结果；在 `shifts` 数据更新后，控制器则会激活 Alphabetizer 模块完成排序。

与传统顺序执行的程序不同，黑板架构的控制器能够根据黑板的状态动态调整执行顺序，这使得系统更加适应复杂任务的求解需求。控制器与知识源模块之间的松耦合关系，也确保了系统具有较高的扩展性和灵活性。

黑板风格实现的设计图如图 4.1 所示。图中展示了黑板作为核心数据存储，以及各知识源

模块通过黑板进行数据交互的方式。控制器通过监视黑板状态动态激活模块，从而实现了灵活、高效的问题求解过程。

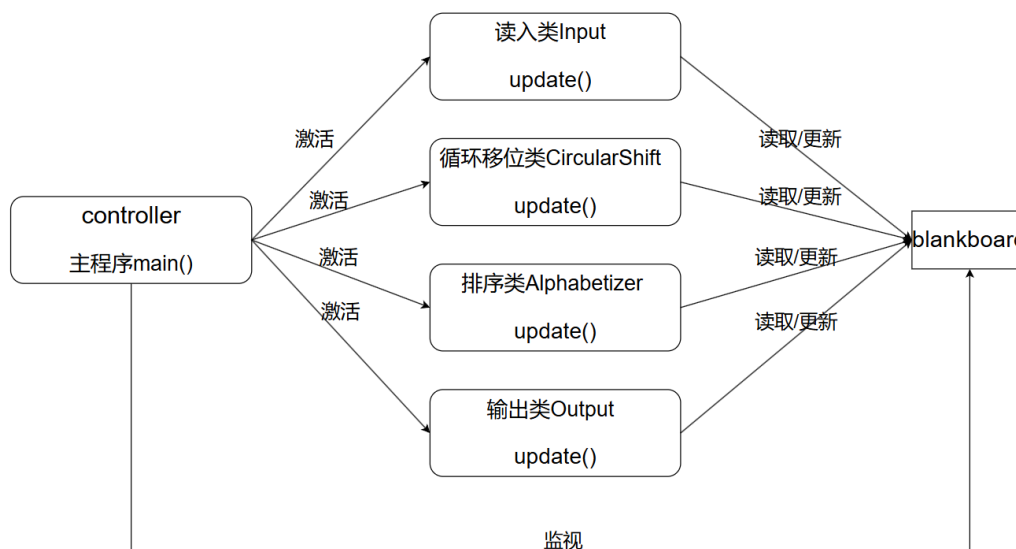


Figure 4.1 黑板风格实现 KWIC 设计图

4.1.1 组件

在黑板风格的设计中，组件是系统功能实现的核心部分，每个组件负责特定的数据处理任务。所有组件通过黑板（Blackboard）进行数据交互，这种设计既确保了功能模块的独立性，又实现了系统整体的协作性。

(1) Blackboard 组件

Blackboard 是系统的核心数据存储组件，用于集中管理程序的所有数据。其数据结构包括用户输入的文本 (`lines`)、循环移位结果 (`shifts`) 和排序后的移位结果 (`sortedShifts`)。例如，当用户输入一组文本行后，Input 模块将这些数据存储到黑板的 `lines` 字段中，其他模块通过读取和更新黑板的数据完成各自的任務。这种集中管理的数据存储方式，既避免了模块之间的直接交互，又提供了统一的数据接口。

(2) Input 组件

Input 模块负责从用户输入中读取数据并存储到黑板的 `lines` 字段中。例如，当用户输入多行文本并以 "END" 表示结束时，程序将这些行存储为一个字符串向量。在代码实现中，`Input::update` 方法通过标准输入逐行读取用户输入，并调用黑板接口存储输入内容，为后续模块提供数据支持。

(3) CircularShift 组件

CircularShift 模块从黑板的 `lines` 字段中读取用户输入的数据，并生成所有可能的循环移位组合，结果存储到 `shifts` 字段中。例如，对于输入行 "hello world"，`CircularShift::update` 方法会生成移位组合 "hello world" 和 "world hello" 并存储到黑板。这一模块实现了循环移位的核心逻辑，其独立性使得循环移位算法可以轻松调整或替换。

(4) Alphabetizer 组件

Alphabetizer 模块从黑板的 `shifts` 字段中读取未排序的循环移位数据，并按照字母顺序对其进行排序，结果存储到 `sortedShifts` 字段中。在实现中，`Alphabetizer::update` 方法利用 `std::sort` 函数对数据进行排序，确保结果的规范性。例如，“Hello World”和“hello world”在排序中被视为相同。

(5) Output 组件

Output 模块从黑板的 `sortedShifts` 字段中读取排序后的循环移位结果，并将其逐行输出到控制台。在代码中，`Output::update` 方法负责读取数据并按行输出。例如，用户可以在程序结束时看到排序后的移位结果列表。这一模块作为数据流的终点，将系统的处理结果以直观的方式展示给用户。

通过上述组件的划分，程序实现了数据输入、循环移位、排序和输出的完整流程。每个组件专注于完成特定的任务，并通过黑板实现数据共享和模块协作，构建了一个高效且松耦合的系统。

4.1.2 连接件

连接件是组件之间数据交互的桥梁，承载了程序中数据流动的核心逻辑。在黑板风格中，各模块通过读取和更新黑板中的字段实现数据的流动和处理，避免了模块之间的直接耦合。

(1) 用户输入的字符串向量 (`lines`)

`lines` 是黑板的一个关键字段，由 Input 模块写入，存储用户逐行输入的文本数据。例如，用户输入“hello world”和“blackboard system”，这些文本被存储为 `lines` 中的独立字符串。其他模块通过读取该字段获取输入数据，作为后续处理的基础。

(2) 循环移位生成的字符串向量 (`shifts`)

`shifts` 是由 CircularShift 模块生成并存储到黑板中的字段，包含用户输入的所有循环移位组合。例如，对于输入行“hello world”，`shifts` 包含“hello world”和“world hello”。这一字段是排序模块的输入数据，承载了移位逻辑的处理结果。

(3) 排序后的字符串向量 (`sortedShifts`)

`sortedShifts` 是 Alphabetizer 模块生成的字段，存储按字母顺序排列的移位结果。例如，对于输入“hello world”和“blackboard system”，排序后的结果可能是“blackboard system”和“hello world”。这一字段被 Output 模块读取并输出到控制台，是数据流的终点。

(4) 黑板的状态数据

黑板中的状态数据不仅包含输入、移位和排序结果，还反映了当前数据处理的进度。例如，当 CircularShift 完成移位处理后，黑板中会增加新的 `shifts` 数据供 Alphabetizer 使用。控制器通过监视黑板状态，动态激活模块以完成数据的流动和处理。

通过连接件的有效设计，程序在各模块之间实现了数据的高效传递和共享。黑板作为全局数据结构，统一管理所有数据，既避免了模块间的直接耦合，又确保了数据流向的清晰性和逻辑一致性。

4.2 流程图说明

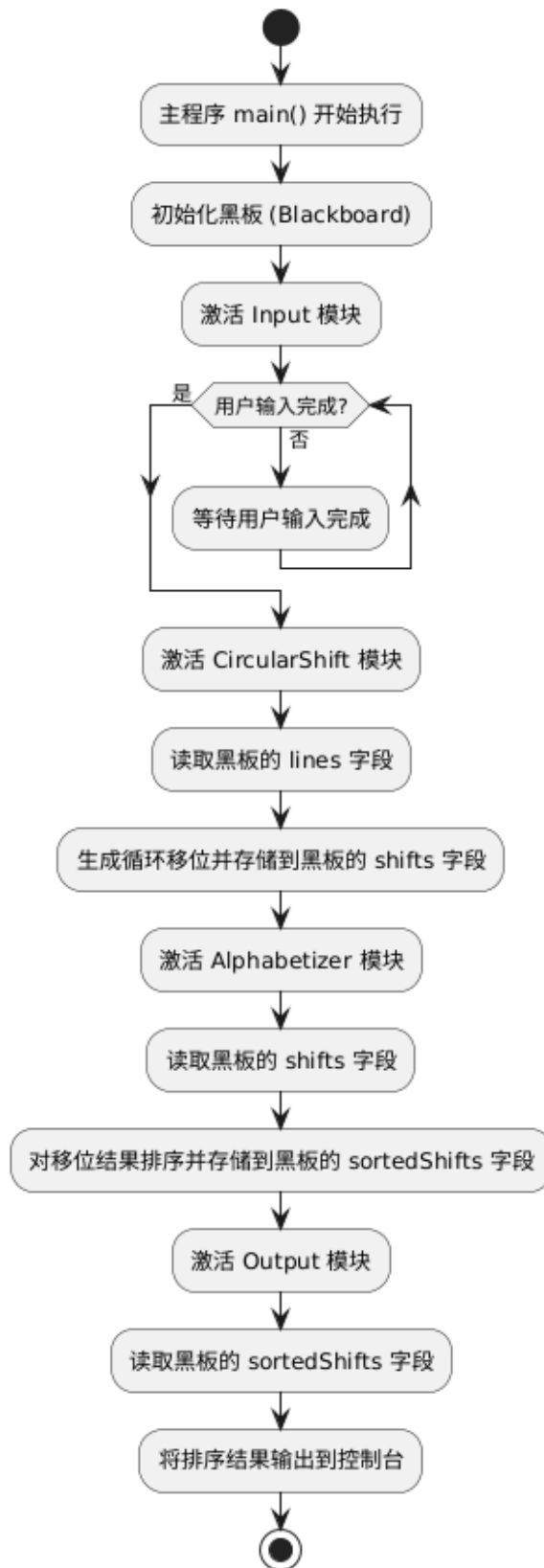


Figure 4.2 黑板风格 KWIC 实现流程图

4.3 结果界面截图

```
Microsoft Visual Studio 调试  x + -
黑板风格实现：
请输入行（输入 'END' 结束）：
hello world
that is ok
the only limit is doubts
END

排序后的循环移位结果：
doubts the only limit is
hello world
is doubts the only limit
is ok that
limit is doubts the only
ok that is
only limit is doubts the
that is ok
the only limit is doubts
world hello

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_Blackboard.exe (进程 11548)已退出，代码为 0。
```

Figure 4.3 黑板风格结果截图

4.4 风格优缺点

4.4.1 优点

1) 便于添加新的知识源模块或扩展黑板数据结构

黑板架构支持动态扩展，通过添加新的模块可以实现功能扩展。例如，若需要在排序前对数据进行过滤，只需新增一个数据过滤模块，并在控制逻辑中激活该模块即可。图 4.4 展示了新增数据过滤模块后的系统设计图。可以看出数据过滤模块作为独立的知识源，与黑板协同工作，从黑板中读取数据并将过滤后的结果存储回黑板。主程序(main)的控制逻辑通过激活该模块，动态调整了数据流向。这种设计支持灵活的系统扩展，减少了开发和维护成本，同时保证了模块间的低耦合性。

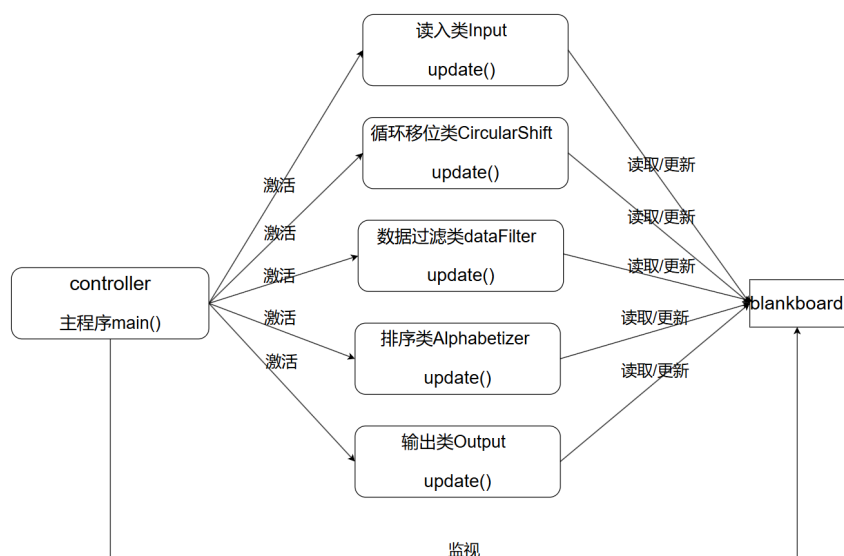


Figure 4.4 黑板风格扩展数据过滤模块后新设计图

2) 便于多模块共享大量数据

黑板架构通过一个共享的数据存储 (Blackboard), 使多个模块可以协作完成任务, 而不需要直接交互。例如, `CircularShift` 模块只需从黑板中读取用户输入的 `lines` 字段, 而无需关心这些数据是如何生成的。类似地, `Alphabetizer` 模块可以直接从黑板的 `shifts` 字段获取数据进行排序, 而无需知道循环移位的生成细节。这种设计使得模块间数据的生产者和消费者完全解耦, 便于系统扩展和维护。

3) 模块复用性强

每个模块独立完成特定功能, 具有高度的复用性。例如, `CircularShift` 模块可以在其他需要循环移位的项目中直接使用, 而无需对代码进行修改。同样, `Alphabetizer` 模块的排序逻辑也可以应用于其他需要排序的场景。模块的独立性和复用性大大提高了代码的开发效率和适应性。

4) 支持容错性和健壮性

黑板架构的松耦合特性使系统具有较高的容错能力。这种设计特性确保了各模块之间的独立性, 使得即使某个模块发生错误, 也不会影响其他模块的正常运行。例如, 在 KWIC 系统中, 如果 `Alphabetizer` 模块的排序逻辑出错, 可能会导致生成的排序结果不符合预期。然而, 这一错误不会影响 `Input` 模块对用户输入的读取, 也不会干扰 `CircularShift` 模块生成循环移位的过程。其他模块仍然能够按照预定流程正常工作。

此外, 在复杂任务处理中, 这种松耦合特性尤其重要。比如, 在实际应用中, 若一个模块需要处理异常输入 (如空行或含有特殊字符的文本), 其错误处理失败只会导致部分数据处理不完整, 而不会影响其他模块的功能。例如, 如果 `CircularShift` 模块未正确处理含有多个连续空格的输入行, 导致生成的移位结果有冗余空格, 这一问题仅体现在 `shifts` 字段中, 而不会影响其他字段的数据完整性。开发者可以在调试时专注于这一模块, 并通过观察黑板中 `shifts` 字段的内容快速定位问题。

4.4.2 缺点

1) 共享数据结构的修改复杂性高:

黑板作为模块间数据交互的核心, 其数据结构的任何修改都会对所有相关模块产生影响。例如, 若需要调整 `lines` 字段的存储格式, 例如从字符串向量 (`std::vector<std::string>`) 改为 JSON 格式, 这种调整将导致多个模块的逻辑需要同步修改。`Input` 模块需要修改将用户输入写入黑板的方式, `CircularShift` 模块需要调整读取 `lines` 字段并解析数据的逻辑, 而其他可能依赖 `lines` 数据的模块也可能受到影响。由于共享数据结构的改动会传递到所有使用该数据的模块, 开发人员需要确保所有相关模块的逻辑一致, 增加了修改和维护的复杂性。

2) 需要同步机制保证数据一致性:

当多个模块同时访问黑板时, 需要同步机制 (如添加互斥锁) 来保证数据的完整性和一致性。例如, 假设 `CircularShift` 模块在生成循环移位时, 还未完全写入 `shifts` 字段的数据, 而 `Alphabetizer` 模块已经开始读取 `shifts` 字段进行排序。这种并发访问可能导致部分循环移位结果缺失或数据不完整, 最终影响排序结果。在高并发场景下, 系统需要通

过加锁机制确保模块之间的访问同步，避免数据冲突。然而，这种同步机制不仅增加了开发难度，还可能引入额外的性能开销，例如线程阻塞或锁竞争。

3) 性能开销较大：

黑板架构中，数据的频繁读取和写入可能导致性能瓶颈。例如，在处理包含数千行输入文本的数据集时，`CircularShift` 模块需要为每一行文本生成大量循环移位，并将结果存储到 `shifts` 字段。此时，`shifts` 字段会占用大量内存空间。同时，`Alphabetizer` 模块需要从黑板读取这些循环移位并对其排序，排序过程中还可能临时创建中间存储，进一步增加内存占用和计算资源消耗。对于大规模数据处理，这种频繁的读取和写入操作可能严重影响系统性能，尤其是在内存有限或计算资源紧张的情况下。

4) 调试和维护难度高：

黑板架构通过共享数据实现模块间的松耦合，但在调试时需要逐步检查数据流动的每个环节，增加了调试的复杂性。例如，如果用户最终获得的排序结果有误，开发者需要逐步排查问题，首先检查 `Input` 模块写入的 `lines` 数据是否正确，然后验证 `CircularShift` 模块生成的 `shifts` 数据是否完整，最后检查 `Alphabetizer` 模块对 `shifts` 数据的排序逻辑。这种逐层检查的调试模式在处理复杂流程或大规模数据时尤为耗时。此外，模块之间通过黑板传递数据的隐式依赖关系，可能在维护时引发意料之外的问题。例如，若某模块新增一个对 `shifts` 字段的处理逻辑，可能无意间干扰了 `Alphabetizer` 模块的读取操作，从而引发系统错误。

4.5 总结

黑板风格通过集中式数据管理和模块化设计，为 KWIC 系统提供了高效、灵活的架构。通过黑板（Blackboard）统一存储系统中的所有数据，各模块专注于完成各自的功能，如输入、循环移位、排序和输出。模块之间通过黑板实现数据交互，避免了直接依赖，提高了系统的灵活性和可维护性。这种设计便于开发人员单独测试和调试模块，同时降低了开发和维护成本。

黑板作为集中式数据存储，为数据流的管理提供了统一的解决方案，避免了模块直接共享数据可能导致的问题。所有数据，包括用户输入的文本、循环移位结果和排序结果，都以统一格式存储在黑板中，确保了数据流的规范性和一致性。然而，由于所有模块都依赖黑板的存储结构，任何数据格式的调整都会对多个模块产生连锁影响，增加了系统维护的复杂性。

尽管黑板架构具有模块化和数据共享的显著优势，但也存在性能和扩展性上的局限性。频繁的数据读写可能导致性能瓶颈，尤其在处理大规模数据时，例如从生成移位到排序的过程中。此外，管理并发访问需要同步机制以确保数据一致性，这进一步增加了系统的复杂性和运行成本。

未来可以通过引入缓存机制减少对黑板的频繁访问，或者采用分布式黑板存储以分散数据流的压力，从而提升系统性能。同时，事件驱动模块激活方式也可以提高系统的灵活性和效率。总体而言，黑板架构适用于多模块协作的数据密集型任务，其优缺点需根据具体需求进行权衡。

第五章 平台/插件风格

5.1 平台/插件风格设计

本章的 KWIC 项目采用了平台/插件风格设计，平台/插件（Platform/Plug-in）风格是一种通过定义统一接口规约，使得程序主体和插件可以动态协作的架构设计。在该架构中，平台承担了系统的核心管理职责，提供插件的加载、管理和执行机制；插件则作为独立的功能模块，通过实现统一的接口规范扩展系统的功能。这样的设计具有高度的灵活性和可扩展性，能够动态适应需求变化，适用于功能复杂、需要多模块协作的系统。

在 KWIC 项目的实现中，平台被定义为插件管理器（PluginManager），它负责协调插件的注册、管理和调用，而插件（如 InputPlugin、CircularShiftPlugin 和 AlphabetizerPlugin 等）则以功能扩展模块的形式，通过接口与平台进行交互。插件管理器通过统一的扩展接口（如 process 方法）调用插件提供的功能，而插件则通过实现预定义的接口（IKWICPlugin）实现与平台的解耦。这种双向交互模式不仅确保了平台与插件之间的独立性，还支持插件的动态加载与卸载，从而满足了系统运行过程中功能调整的需求。

本架构将核心功能划分为两个部分：程序的主体（平台）和功能扩展模块（插件）。平台实现了插件的管理机制，包括插件的动态注册与调用；插件则专注于完成具体的功能任务，如数据的读取、处理和输出。通过这一分工，平台提供了系统运行的基础能力，而插件通过平台提供的扩展接口对其进行功能增强。例如，CircularShiftPlugin 插件提供了对输入文本的循环移位功能，而 AlphabetizerPlugin 插件则对移位结果按字母顺序排序。插件之间通过平台实现了功能的协作和数据流的动态管理。

如图 5.1 所示，平台与插件之间通过扩展接口（Platform Extension Interface）和插件接口（Plug-in Interface）进行通信。平台通过扩展接口为插件提供系统资源，如共享数据结构 lines，同时调用插件实现的功能接口以完成具体的处理任务。这种设计模式实现了平台与插件之间的松耦合，降低了模块间的依赖性，使得系统的功能扩展和维护更加高效。

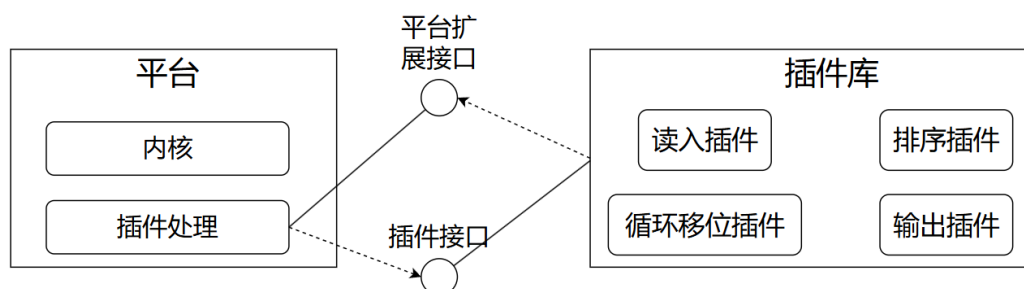


Figure 5.1 平台插件风格实现 KWIC 设计图

平台插件架构的优势不仅体现在其灵活的扩展能力上，还表现为其模块化的设计思想。

插件开发者只需关注插件接口的实现，无需关心平台的内部逻辑，这使得插件可以独立开发、测试和部署。例如，在本代码中，`InputPlugin` 插件负责从用户输入中读取数据，而无需了解其他插件的实现细节。同样，`OutputPlugin` 插件只需处理最终的输出数据，而不需要关心数据的来源和加工过程。这样的模块化设计提高了系统的开发效率和维护性，同时也为第三方开发者提供了便捷的扩展途径。

平台插件风格通过清晰的模块分工、统一的接口规范以及动态的功能扩展能力，为系统设计提供了一种高效、灵活的解决方案。它特别适合于需要频繁功能升级或扩展的复杂系统，同时也为功能模块的独立开发和协作提供了强有力的支持。

5.1.1 组件

平台插件风格的核心在于平台与插件的协同工作，以下是系统中主要组件的详细说明：

(1) 插件接口 (`IKWICPlugin`)

插件接口是平台与插件交互的桥梁，定义了插件必须实现的标准方法。在本系统中，所有插件都实现了 `IKWICPlugin` 接口的 `process` 方法，用于对共享数据结构 `lines` 进行处理。通过这种统一接口，平台能够以一致的方式调用不同插件的功能，而无需关心插件的具体实现细节。例如，`CircularShiftPlugin` 和 `AlphabetizerPlugin` 都通过实现 `process` 方法，在各自职责范围内对数据进行变换和排序。

(2) 插件管理器 (`PluginManager`)

插件管理器是平台的核心组件，负责管理插件的动态加载和顺序执行。开发者可以通过 `addPlugin` 方法将插件添加到管理器中，随后通过 `executePlugins` 方法依次调用所有插件。例如，系统启动时，`InputPlugin` 被首先调用以收集用户输入，之后 `CircularShiftPlugin` 和 `AlphabetizerPlugin` 对数据进行处理，最终由 `OutputPlugin` 展示结果。插件管理器隐藏了插件的具体实现，使平台具备高度的模块化和扩展能力。

(3) 输入插件 (`InputPlugin`)

输入插件作为数据流的起点，负责从用户处读取多行输入文本，并将其存储到共享的数据结构 `lines` 中。输入的终止条件是用户输入“END”，这避免了用户误操作导致的无效数据进入系统。输入插件通过实现 `IKWICPlugin` 接口的 `process` 方法，封装了数据采集的逻辑，确保后续插件能够接收到格式化的输入数据。

(4) 循环移位插件 (`CircularShiftPlugin`)

循环移位插件读取 `lines` 中的文本，并对每一行生成所有可能的循环移位组合，结果存储回 `lines` 中。它是系统处理逻辑的核心部分，通过对文本的词序重新排列，生成丰富的组合数据。例如，对于输入“hello world”，循环移位插件会生成“hello world”和“world hello”两种组合。这种操作为后续的排序和输出提供了多样化的基础数据。

(5) 排序插件 (`AlphabetizerPlugin`)

排序插件对 `lines` 中的循环移位结果按照字母顺序进行排序，并覆盖原有内容。排序逻辑采用标准的 `std::sort` 算法，确保数据以字母表顺序排列。例如，对于输入“world hello”和“hello world”，排序插件会输出“hello world”和“world hello”。这一过程规范了数据的展示格式，使最终结果更具可读性。

(6) 输出插件 (OutputPlugin)

输出插件是数据流的终点，负责将处理完成的 `lines` 数据逐行输出到控制台。通过调用 `process` 方法，输出插件读取 `lines` 中的排序结果，并格式化打印到用户界面。例如，对于输入“hello world”，经过处理后，输出插件会打印“hello world”和“world hello”。这一设计实现了数据从输入到输出的完整闭环。

通过上述组件的分工与协作，系统实现了输入、循环移位、排序和输出的功能链。每个模块功能独立且接口统一，大幅提升了系统的扩展性和可维护性。

5.1.2 连接件

平台插件风格中的连接件定义了平台与插件之间的交互方式，以及插件之间通过共享数据结构进行的数据流动。以下是系统中主要连接件的详细说明：

(1) 平台与插件的接口交互

平台通过调用插件实现的 `process` 方法，与插件实现单向通信。每个插件通过实现 `IKWICPlugin` 接口的 `process` 方法定义了自己的功能逻辑，而平台只需通过接口调用插件，无需关心插件的内部实现。例如，`PluginManager` 的 `executePlugins` 方法会逐个调用已注册插件的 `process` 方法，从而完成一系列功能处理。

(2) 共享的数据结构 `lines`

`lines` 是插件间数据交互的核心存储，所有插件通过对 `lines` 的读取和更新，完成系统的数据流转。例如，`InputPlugin` 将用户输入的数据写入 `lines`，`CircularShiftPlugin` 对 `lines` 中的内容进行变换，`AlphabetizerPlugin` 对其进行排序，而 `OutputPlugin` 则读取并展示结果。这种共享数据结构避免了模块之间的直接依赖，保证了系统的松耦合特性。

(3) 插件的顺序调用

插件管理器按照注册顺序依次调用所有插件，保证了功能处理的逻辑链。例如，`InputPlugin` 的输出作为 `CircularShiftPlugin` 的输入，移位结果再被传递到 `AlphabetizerPlugin` 进行排序，最终由 `OutputPlugin` 展示。这种顺序调用通过平台实现了模块间的动态连接，使系统的功能逻辑清晰直观。

(4) 插件的动态注册与执行

插件通过 `addPlugin` 方法动态注册到平台中，运行时可动态添加或移除插件。插件管理器在调用时无需了解插件的具体实现逻辑，只需通过接口与其交互。例如，可以在运行时添加一个新的数据过滤插件，而无需对已有的插件或平台进行任何修改。这种设计实现了系统功能的动态扩展和灵活组合。

平台插件风格通过清晰的接口规范和共享数据结构，实现了系统模块化和动态扩展的目标。插件间松耦合的特性使得功能扩展和维护变得更加高效，适合复杂系统的开发。

5.2 流程图说明

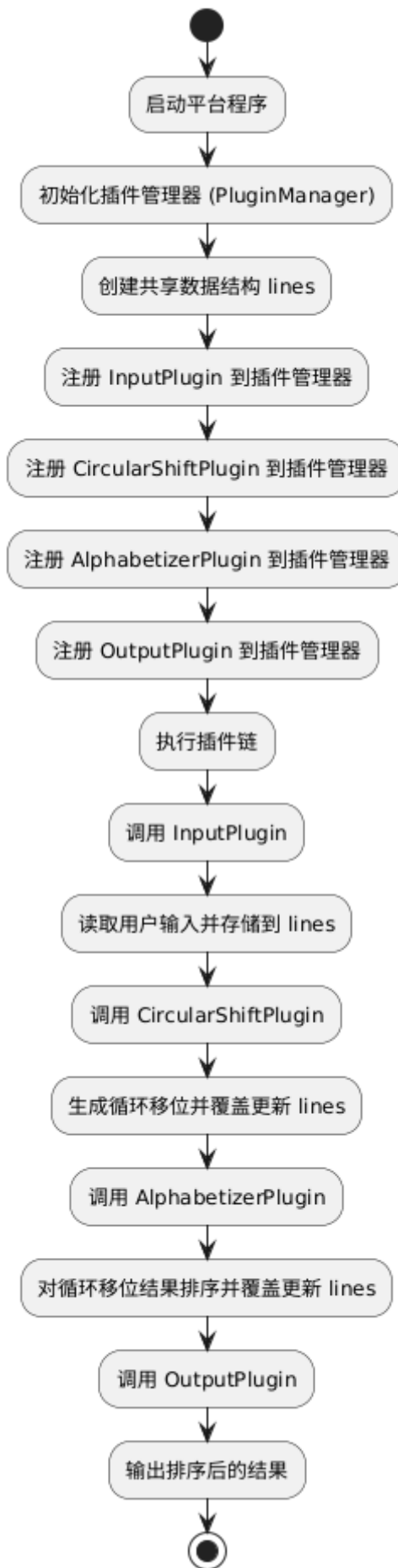
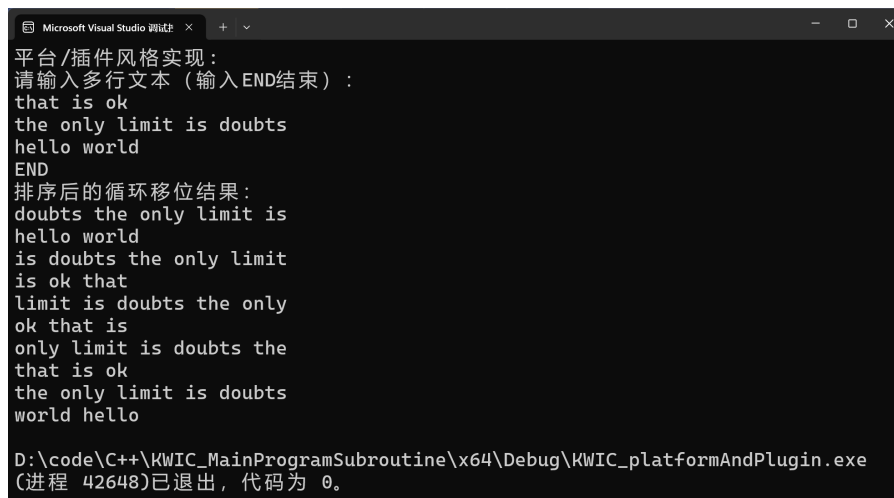


Figure 5.2 平台插件风格 KWIC 实现流程图

5.3 结果界面截图



```
Microsoft Visual Studio 调试
平台/插件风格实现：
请输入多行文本（输入END结束）：
that is ok
the only limit is doubts
hello world
END
排序后的循环移位结果：
doubts the only limit is
hello world
is doubts the only limit
is ok that
limit is doubts the only
ok that is
only limit is doubts the
that is ok
the only limit is doubts
world hello

D:\code\C++\KWIC_MainProgramSubroutine\x64\Debug\KWIC_platformAndPlugin.exe
(进程 42648)已退出，代码为 0。
```

Figure 5.3 平台插件风格结果截图

5.4 风格优缺点

5.4.1 优点

(1) 模块化设计降低了系统耦合度

平台插件风格通过统一的接口规范（如 `process` 方法），实现了平台与插件的松耦合。插件管理器（`PluginManager`）作为平台的核心，通过动态加载插件和调用其功能，避免了插件之间的直接依赖。例如，`CircularShiftPlugin` 和 `AlphabetizerPlugin` 的功能完全独立，不会相互影响。这种模块化设计便于系统功能的扩展和维护。

(2) 支持插件的独立开发、部署与维护

插件是通过实现统一接口规范而独立开发的模块，开发者可以单独开发、测试和部署插件，而不影响平台或其他插件的运行。例如，`InputPlugin` 专注于读取用户输入，而 `OutputPlugin` 专注于输出处理结果。这种设计使得插件的开发流程更加灵活，同时大幅降低了系统维护成本。

(3) 灵活的动态扩展能力

插件管理器支持插件的动态加载和执行，允许系统在运行时根据需求调整功能。例如，如果需要新增数据过滤功能，只需开发一个过滤插件并通过 `addPlugin` 方法注册到平台中，无需修改原有代码。这种动态扩展能力使得系统能够快速响应需求变化，提高了灵活性。

(4) 提升了系统的功能复用性

插件的独立性和标准化接口设计使得其功能可以在不同项目中复用。例如，`AlphabetizerPlugin` 的排序逻辑不仅可以用于 KWIC 系统，还可以应用于其他需要数据排序的场景。通过统一的接口规范，插件可以在不同平台上实现跨项目的高效复用。

5.4.2 缺点

(1) 插件的重用性受限于特定平台

插件的实现通常依赖于特定平台的接口规范，使得插件通用性较差。例如，`InputPlugin` 和 `CircularShiftPlugin` 是针对 `PluginManager` 平台设计的，无法直接应用于其他平台。如果需要跨平台使用插件，开发者需要进行额外的适配工作，增加了开发成本。

(2) 动态加载增加了运行时开销

插件的动态加载和执行带来了额外的性能开销。在处理大规模数据时，插件间的多次数据传递和处理可能成为系统的性能瓶颈。例如，`CircularShiftPlugin` 会生成大量循环移位结果，并将其传递给 `AlphabetizerPlugin` 进行排序，这种频繁的数据操作可能导致内存占用和计算开销增加。

(3) 对接口设计的要求较高

平台插件风格依赖于统一的接口规范，接口设计的复杂性直接影响插件与平台的协作效率。如果接口规范不完善或缺乏清晰文档，可能导致插件实现与平台期望不一致。例如插件未正确实现 `process` 方法的功能逻辑，可能导致系统运行失败或输出错误结果。

(4) 调试和维护的复杂性

插件的动态加载和执行增加了系统的调试难度。在插件链较长或功能复杂的情况下，错误的定位需要逐步检查每个插件的输入、处理逻辑和输出。例如，若最终输出结果有误，开发者需要依次检查 `InputPlugin`、`CircularShiftPlugin` 和 `AlphabetizerPlugin` 的执行过程，调试成本较高。

5.5 总结

平台插件风格以其松耦合和接口规范的特性，为系统的动态扩展和模块化设计提供了强有力的支持。这种架构通过明确的接口定义，实现了平台与插件之间的独立性，极大地降低了模块间的依赖性。各插件可以独立开发、部署和维护，而无需对平台或其他插件的功能产生影响。同时，平台插件架构允许系统根据实际需求动态地加载、卸载或调整插件，从而具备高度的灵活性，能够快速响应功能扩展的需求。

然而，尽管平台插件风格在灵活性和模块化方面表现出色，其在实际应用中 also 面临一定的挑战。例如，由于插件的实现通常与特定平台紧密耦合，其跨平台的复用性较为有限，这可能增加插件在不同项目中重复开发的成本。此外，插件的动态加载和执行可能带来一定的性能开销，特别是在处理大量数据或复杂逻辑时，这种性能损耗可能成为系统运行的瓶颈。接口规范的设计也对系统的成功至关重要，不完善的接口定义可能导致平台与插件之间的功能协作不一致，甚至引发系统运行错误。

因此，在实际应用平台插件风格时，开发者需要权衡其灵活性与复杂性之间的关系，应在设计阶段明确接口规范，确保插件和平台之间的协作一致性。同时，通过优化插件的管理机制，减少性能开销，提升系统效率。在插件设计上，也应尽可能提高通用性，以便在不同平台中实现高效复用。

通过合理的规划与优化，平台插件风格不仅能够满足复杂系统动态扩展和模块化的需求，还能在开发效率和系统性能之间实现良好的平衡，为现代软件架构提供坚实的技术支撑。

第六章 五种风格对比

在系统设计中，不同的架构风格适用于不同的应用场景，并具有各自的优势和不足。在 KWIC（Key Word in Context）项目中，不同架构风格表现出各自的特性，以下是主程序-子程序风格、批处理风格、面向对象风格、黑板风格和平台/插件风格的特性比较。

6.1 KWIC 项目中的架构风格比较

1) 模块化：

模块化是架构风格设计的核心衡量指标之一，其直接影响系统的维护性、开发效率和扩展能力。在 KWIC 实现中，各种风格的模块化程度体现如下：

- **主程序-子程序风格：**将系统的功能拆分为多个子程序，每个子程序负责特定的任务。例如，在 KWIC 中，输入、循环移位、排序和输出被设计为独立的函数，通过主程序来控制调用。这种风格的模块化程度较低，模块之间通过参数紧耦合，修改某一功能时需同步调整其他模块，易引入复杂性，但对简单项目开发有较高效率。
- **批处理风格：**将系统分解为若干过滤器模块，每个模块独立完成特定任务并通过数据流传递数据。例如，输入模块负责读取数据并传递给移位模块，移位模块的输出再传递给排序模块。模块间完全松耦合，新增或替换某个过滤器模块无需修改其他模块，体现了较高的模块化水平。
- **面向对象风格：**采用类和对象封装功能，职责分配清晰。例如，Input 类用于封装输入逻辑，CircularShift 类负责移位操作，Alphabetizer 类实现排序功能。通过接口设计实现模块间的解耦，支持独立开发和复用，模块化程度较高。
- **黑板风格：**以全局共享的黑板存储作为模块间协作的媒介。例如，KWIC 的黑板可能包含共享的 lines 和 shifts 数据结构，所有模块通过读取和更新黑板数据进行协作。模块间无直接耦合，具有较高的模块化程度，但黑板结构的设计对系统整体运行至关重要。
- **平台/插件风格：**将功能设计为独立插件模块，通过标准接口与平台交互。例如，InputPlugin 和 OutputPlugin 插件负责处理输入和输出逻辑，而平台只需动态加载插件即可。这种风格模块化程度最高，各插件可独立开发和部署，且不影响其他功能模块。

通过上面对每种风格模块化程度的说明，可以看出主程序-子程序风格模块化程度较低，模块间紧耦合，适合简单功能；批处理风格模块松耦合，各功能通过数据流独立协作；面向对象风格以类封装功能，职责分明，支持复用；黑板风格依赖全局共享存储，模块协作

高效；平台/插件风格模块独立性最强，可动态扩展。

2) 灵活性与扩展性:

灵活性与扩展性是指系统面对需求变化时的适应能力。在 KWIC 实现中，这些风格的表现如下：

- **主程序-子程序风格**：扩展性较差，新增功能（如数据过滤）需要直接修改主程序的控制逻辑，可能导致代码冗长且维护困难。
- **批处理风格**：新增功能只需插入新的过滤器模块，例如在输入模块和移位模块之间插入数据过滤模块。无需更改现有模块，扩展性高。
- **面向对象风格**：支持通过继承和多态机制实现功能扩展。例如，可以通过继承 `Alphabetizer` 类实现自定义排序逻辑。但如果扩展涉及多个模块之间的交互，可能需要调整类之间的关联，扩展复杂功能的成本较高。
- **黑板风格**：灵活性较高，可以通过新增知识源模块实现功能扩展。然而，如果黑板的数据结构需要改变，则所有依赖于黑板的模块都需同步调整，维护难度增加。
- **平台/插件风格**：扩展性最强，通过动态加载和卸载插件实现功能扩展。例如，可以通过注册一个新的插件实现对特定格式的输入文件支持，整个过程无需停止系统运行。

通过上面对每种风格灵活性与扩展性的说明，可以看出主程序-子程序风格扩展性较差，新增功能需修改逻辑；批处理风格扩展性高，可插入模块独立实现；面向对象风格支持继承与多态扩展，但复杂交互调整成本较高；黑板风格灵活性强，扩展需同步数据结构；平台/插件风格扩展性最强，支持动态加载与运行。

3) 数据流与交互:

数据流的设计决定了系统的运行效率和模块协作方式。在 KWIC 的实现中，数据流的特点如下：

- **主程序-子程序风格**：数据流由主程序集中控制，各子程序通过参数显式传递数据，流程简单直观，但不适合复杂数据处理。
- **批处理风格**：数据以流的形式通过数据流在过滤器之间传递。每个模块只需关注输入和输出，易于扩展，适合顺序性强的数据处理任务。
- **面向对象风格**：数据通过方法调用进行传递。支持复杂交互，但模块间可能产生隐式依赖，需要谨慎设计接口。
- **黑板风格**：数据集中存储在全局共享的黑板中，模块通过黑板读取和更新数据。这种方式支持异步操作，适用于复杂协作性强的系统。
- **平台/插件风格**：数据通过标准接口传递，插件只需实现接口方法即可完成与平台的数据交互，插件之间无直接依赖，数据流松耦合。

通过上面对每种风格数据流与交互的说明，可以看出主程序-子程序风格数据流集中控制，参数显式传递；批处理风格数据流以数据流形式顺序传递，模块关注单一；面向对象风格通过方法调用传递数据，支持复杂交互但可能有隐式依赖；黑板风格采用共享黑板进行数据交互，支持异步操作；平台/插件风格通过标准接口传递数据，插件间数据流松耦合且无直接依赖。

4) 性能开销:

不同架构风格的性能开销与其模块设计和数据交互方式密切相关:

- **主程序-子程序风格:** 因模块紧耦合且数据传递直接, 性能开销最低。这种风格依赖主程序对功能的集中控制, 通过直接的参数传递实现数据流动, 避免了额外的中间处理开销。然而, 当函数调用频繁或系统规模增大时, 可能出现效率下降的情况, 尤其在复杂计算场景中可能成为瓶颈。
- **批处理风格:** 由于数据在数据流间多次复制, 内存占用较高, 在处理大规模数据时性能下降。这种风格通过将功能分解为多个过滤器模块实现松耦合, 但数据在过滤器间的传递需要不断复制, 导致内存和计算资源消耗增加, 尤其在处理高吞吐量或大数据任务时表现出一定的性能限制。
- **面向对象风格:** 类的封装和对象管理带来了额外的内存开销, 但代码逻辑复杂时更易维护。这种风格通过封装逻辑、分配职责和提供接口实现功能, 尽管性能略有牺牲, 但带来了良好的代码组织和可维护性, 特别适合功能复杂且需要长期迭代的系统。
- **黑板风格:** 模块通过共享的黑板读写数据, 频繁的同步和数据更新增加了性能负担, 尤其在并发环境下开销更大。虽然这种风格适用于多模块协作的复杂场景, 但共享黑板的设计对性能影响显著, 尤其当数据更新和访问频率较高时, 同步机制会成为性能瓶颈。
- **平台/插件风格:** 动态加载插件和接口调用增加了运行时开销, 但以此换取了更高的灵活性。这种风格通过支持插件的动态加载与卸载, 实现了系统功能的灵活扩展, 尽管运行时的管理和接口调用增加了额外开销, 但在对适应性和扩展性要求较高的场景中仍然表现出优越性。

通过上面对每种风格性能开销的说明, 可以看出主程序-子程序风格性能开销最低, 因数据传递直接且模块紧耦合; 批处理风格性能受数据复制影响, 在大规模数据处理时内存占用较高; 面向对象风格因类封装和对象管理带来额外开销, 但维护性较好; 黑板风格频繁的黑板数据同步和更新增加了性能负担; 平台/插件风格动态加载和接口调用增加运行时开销, 但灵活性显著提升。

- 5) **时间消耗:** 在对五种架构风格实现 KWIC 的过程中, 为了保证测试结果的准确性, 将输入内容固定, 避免用户输入或运行环境的不确定性影响测试时间。随后, 对每种架构风格的实现分别运行多次, 并取其平均值作为时间消耗的参考。通过测试, 得到了五种风格在相同功能实现下的时间消耗结果, 具体如图 6.1 所示。

主程序-子程序风格实现:	批处理风格实现:	面向对象风格实现:	黑板风格实现:	平台/插件风格实现:
排序后的循环移位结果: world hello the only limit is doubts that is ok only limit is doubts the ok that is limit is doubts the only is ok that is doubts the only limit hello world doubts the only limit is Execution time: 776 microseconds	排序后的循环移位结果: doubts the only limit is hello world is doubts the only limit is ok that limit is doubts the only ok that is only limit is doubts the that is ok the only limit is doubts world hello Execution time: 937 microseconds	排序后的循环移位结果: world hello the only limit is doubts that is ok only limit is doubts the ok that is limit is doubts the only is ok that is doubts the only limit hello world doubts the only limit is Execution time: 1027 microseconds	排序后的循环移位结果: doubts the only limit is hello world is doubts the only limit is ok that limit is doubts the only ok that is only limit is doubts the that is ok the only limit is doubts world hello Execution time: 1154 microseconds	排序后的循环移位结果: doubts the only limit is hello world is doubts the only limit is ok that limit is doubts the only ok that is only limit is doubts the that is ok the only limit is doubts world hello Execution time: 1195 microseconds

Figure 6.1 五种风格时间消耗对比

- **主程序-子程序风格**：执行时间为 776 微秒，表现出最高的效率。这种风格采用紧凑的结构设计，所有逻辑由主程序直接控制，减少了模块间的通信开销，适合对性能要求较高但功能逻辑较简单的场景。
- **批处理风格**：执行时间为 937 微秒，效率次之。这种风格通过数据流连接各个过滤器模块，尽管模块间松耦合，但数据的多次复制和传递增加了内存占用，导致性能稍有下降。
- **面向对象风格**：执行时间为 1027 微秒。由于面向对象风格需要通过类的封装和对象的管理来实现功能，尽管提高了模块的复用性和可扩展性，但类实例化和方法调用的开销导致了时间消耗略高。
- **黑板风格**：执行时间为 1154 微秒。黑板风格通过共享黑板实现模块间的协作，支持异步操作和动态数据交互，但频繁的黑板读写和同步机制增加了性能负担，在处理大规模数据时效率相对较低。
- **平台/插件风格**：执行时间为 1195 微秒，在五种风格中时间消耗最高。这种风格支持动态加载和卸载插件，提供了极高的灵活性和扩展性，但接口调用、插件加载和运行时的管理开销显著增加了执行时间。

上述结果表明，随着架构设计的灵活性和扩展性增强，系统的性能开销也随之增加。主程序-子程序风格执行时间最短，仅 776 微秒，因其结构紧凑，逻辑直接控制，通信开销最低；批处理风格以 937 微秒的时间效率次之，但数据多次复制和传递增加了内存占用；面向对象风格需 1027 微秒，受类封装和方法调用开销的影响，时间消耗略高；黑板风格耗时 1154 微秒，因频繁的黑板读写和同步机制降低了效率；平台/插件风格时间最长，为 1195 微秒，其动态加载插件和接口调用的开销显著增加了执行时间。

6) 适用场景：

- 主程序-子程序风格适用于功能固定、流程简单的小型项目，如教学演示。这种风格以集中控制为特点，能够快速实现核心功能，开发和调试流程简单明了，特别适合需求明确且变化较少的场景。
- 批处理风格适合处理流水线任务或分阶段处理的场景，如日志处理、文本解析等。通过模块间的数据流传递实现功能分离，各模块只需关注单一任务，使得这种风格在扩展新功能或调整流程时灵活性较高，特别适合结构化的任务处理。
- 面向对象风格适用于功能复杂、需高复用性的系统，如通用型文本处理框架。通过类和对象封装功能，职责清晰且支持继承和多态，使得这种风格适合需要灵活扩展和模块独立开发的系统，尤其是代码可维护性和复用性要求较高的场景。
- 黑板风格适合需要模块协作且任务复杂的场景，如多知识源交互的智能系统。这种风格通过全局共享的黑板实现模块间的动态协作，支持异步操作和并发任务，尤其适用于需要多个模块高效协同处理信息的复杂系统。
- 平台/插件风格最适合需求频繁变动的系统，如动态内容管理平台或可扩展性要求高的开发工具。通过动态加载和卸载插件实现功能扩展，能够快速适应需求变化，特别适合面向用户的系统，满足功能模块化和灵活性并重的场景。

通过上面对每种风格适用场景的说明，可以看出五种架构风格在适用场景上各有侧重。主程序-子程序风格适合功能固定、流程简单的小型系统，如教学演示或工具程序；批处理风格以模块松耦合为特点，适用于流水线任务或分阶段处理的场景，如日志处理和文本解析；面向对象风格以类和对象的封装为核心，适合需要高复用性和复杂逻辑处理的系统，如通用文本处理框架；黑板风格依托共享黑板实现模块间的动态协作，适合需要高协作性和异步操作的复杂系统，如多知识源交互的智能系统；平台/插件风格通过动态加载和卸载插件实现高度灵活性，最适用于需求频繁变化的系统，如动态内容管理平台或需要高扩展性的开发工具。

6.2 特性汇总表

根据上一节所述的详细特性分析，可将内容汇总如下表所示：

Table 6.1 KWIC 项目中五种架构风格特性对比

特性	主程序-子程序	批处理	面向对象	黑板	平台/插件
模块化	紧耦合	松耦合	职责清晰	协作高效	独立插件
扩展性	差	高	灵活	灵活但同步调整	动态扩展
数据流	参数传递	数据流	方法调用	黑板共享	接口共享
性能开销	最低	内存占用高	开销适中	同步开销大	动态加载开销
时间消耗 (微秒)	776	937	1027	1154	1195
适用场景	小型系统	流水线任务	复杂系统	模块协作	动态平台

6.3 总结

在软件系统设计中，不同的架构风格通过其特定的设计原则和模块化方式展现出各自的优势与限制。在 KWIC 项目中，五种架构风格在模块设计、灵活性、扩展性、性能表现和时间消耗等方面均体现出显著的差异。这些风格从简单到复杂，从紧耦合到松耦合，从静态实现到动态扩展，展现了系统架构设计的多样性与灵活性。

简单的架构风格更适合功能明确、逻辑固定的场景，具有较低的实现成本和性能开销，但在面对需求变化时往往缺乏足够的灵活性和扩展能力。而复杂的架构风格则通过模块间的松耦合和标准化接口设计，提供了更强的适应性和动态扩展能力，但通常以更高的运行时开销和实现复杂性为代价。

随着系统功能需求的增强和复杂度的提高，架构风格的选择愈发需要在性能、灵活性和扩展性之间进行权衡。实际项目中，不同场景下对性能和灵活性的优先级不同，决定了架构风格的适用性。对于一些高性能要求的场景，较为简单的架构设计更为适宜；而对于需求频繁变化或功能不断扩展的项目，灵活性和扩展性更强的架构设计则显得尤为重要。

通过对 KWIC 项目的不同风格实现测试可以发现，不同架构风格不仅体现了设计理念的差异，也展示了系统在实现过程中的多种权衡。选择合适的架构风格需要深入理解具体需求，

充分评估项目规模、功能复杂度、未来扩展性以及性能要求。结合实际场景，合理设计系统架构，可以有效提升系统的开发效率和运行效能，同时减少后续维护的复杂性。

总的来说，架构设计不仅仅是技术实现的选择，更是系统开发中功能、性能与灵活性的综合平衡。通过针对具体项目的需求合理应用不同的架构风格，可以为系统实现带来更大的灵活性和适应性，从而更好地满足项目目标。